Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3736751

PDF Download
3736751.pdf
23 January 2026
Total Citations: 9
Total Downloads: 2179

Published: 12 July 2025
Online AM: 22 May 2025
Accepted: 03 May 2025
Revised: 01 May 2025
Received: 17 January 2024

Citation in BibTeX format

SURVEY

Maintainability and Scalability in Machine Learning: Challenges and Solutions

KARTHIK SHIVASHANKAR, University of Oslo, Oslo, Oslo, Norway

GHADI AL HAJJ, University of Oslo, Oslo, Oslo, Norway

ANTONIO MARTINI, University of Oslo, Oslo, Oslo, Norway

Open Access Support provided by:

University of Oslo

Maintainability and Scalability in Machine Learning: Challenges and Solutions

KARTHIK SHIVASHANKAR, Department of Informatics, University of Oslo, Oslo, Norway
GHADI AL HAJJ, Department of Informatics, University of Oslo, Oslo, Norway
ANTONIO MARTINI, Informatics, University of Oslo, Oslo, Norway

Rapid advancements in Machine Learning (ML) introduce unique maintainability and scalability challenges. Our research addresses the evolving challenges and identifies ML maintainability and scalability solutions by conducting a thorough literature review of over 17,000 papers, ultimately refining our focus to 124 relevant sources that meet our stringent selection criteria. We present a catalogue of 41 Maintainability and 13 Scalability challenges and solutions across Data, Model Engineering and the overall development of ML applications and systems. This study equips practitioners with insights on building robust ML applications, laying the groundwork for future research on improving ML system robustness at different workflow stages.

CCS Concepts: • **Computing methodologies** → **Machine learning**; **Intelligent agents**; • **Software and its engineering** → **Software development techniques;**

Additional Key Words and Phrases: Machine learning, deep learning, maintainability, scalability

ACM Reference Format:
Karthik Shivashankar, Ghadi Al Hajj, and Antonio Martini. 2025. Maintainability and Scalability in Machine Learning: Challenges and Solutions. *ACM Comput. Surv.* 57, 12, Article 318 (July 2025), 36 pages. https://doi.org/10.1145/3736751

1 Introduction

Machine Learning (**ML**) systems play a crucial role in extracting valuable insights and automating decision-making processes from vast amounts of data. Their capabilities in pattern recognition, prediction, and optimization are unmatched. However, as these systems become more intricate and expansive, they encounter challenges that threaten their long-term viability and effectiveness [2, 24].

The widespread adoption of ML has exposed various critical concerns. Model staleness, stemming from shifts in data distributions over time, compromises the dependability of deployed systems. Training-serving skew, arising from differences between training and production environments, can lead to unexpected and potentially detrimental model behavior [40]. Furthermore, **ML systems** (**MLSs**) accrue technical debt as they evolve, making future modifications and improvements increasingly challenging and costly [1, 39].

Authors' Contact Information: Karthik Shivashankar, Department of Informatics, University of Oslo, Oslo, Norway; e-mail: karths@uio.no; Ghadi Al Hajj, Department of Informatics, University of Oslo, Oslo, Norway; e-mail: ghadia@uio.no; Antonio Martini, Informatics, University of Oslo, Oslo, Norway; e-mail: Antonima@uio.no.

This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).
ACM 0360-0300/2025/07-ART318
https://doi.org/10.1145/3736751

Ensuring data quality, reproducibility of ML experiments, efficient resource management, and effective monitoring and debugging also present ongoing challenges [40].

Two fundamental aspects deserve special attention due to their widespread impact and crucial significance: maintainability and scalability. Maintainability ensures the ability to update, adapt, and sustain ML systems over time, ensuring they remain accurate, reliable, and aligned with evolving business needs [15, 25, 26]. Scalability addresses the ability to handle increasing amounts of data, computational demands, and complexity without compromising performance or efficiency [6, 14, 36].

We chose to focus on these aspects for several compelling reasons. First, they are foundational for the long-term success of ML systems across various domains and applications [24, 47]. Second, they permeate every stage of the ML lifecycle, cutting across different ML paradigms and application domains. Third, they often interact and amplify each other, creating complex problems demanding holistic solutions [38, 40].

Industry practitioners consistently highlight these aspects as major pain points, and despite widespread recognition, there's a notable lack of comprehensive literature systematically addressing them, creating a significant research gap [14, 40].

The relationship between scalability and maintainability in ML systems is often characterized by inherent tension, where improvements in one dimension can impede the other. Designing systems for extreme scalability frequently involves adopting complex architectures, such as distributed systems or microservices [43]. While enabling growth, this complexity introduces significant maintainability challenges, making systems harder to understand, debug across components (a notoriously difficult task in distributed environments [40]), and manage overall. It increases coordination overhead between potentially separate teams or components, can lead to complex configuration management or "configuration debt" [38], and complicates the testing process. Conversely, prioritizing maintainability through simpler, potentially monolithic designs may encounter scaling limitations later as demands grow [43], as vertical scaling has inherent limits. Furthermore, dedicating significant upfront effort to maintainability features like modularity, thorough documentation, and robust testing frameworks can slow initial development velocity [17]. These features (including logging, versioning, and monitoring) also introduce resource overhead, potentially impacting performance metrics needed for scalability.

Ignoring the crucial balance between these competing concerns leads inevitably to technical debt, representing the long-term cost of rework necessitated by choosing easier, short-term solutions [17, 38]. This debt can manifest whether scalability is pursued without adequate focus on maintainability (e.g., through extensive "glue code," creating system "entanglement," or neglecting documentation and testing) or if overly simplistic designs fail to anticipate future scaling requirements [17]. ML systems are susceptible to specific forms of technical debt arising from data dependencies, model entanglement, hidden feedback loops, and intricate configuration complexity, often residing at the system or data level and requiring holistic monitoring approaches to effectively manage [17, 38].

This fundamental tradeoff permeates the entire ML system development lifecycle, impacting interconnected stages like data engineering, model engineering, and broader system-level concerns [30, 45]. The stages interact, creating complex challenges; for instance, poor data quality (a data engineering issue) affects model accuracy (requiring maintenance), while inefficient resource management impacts model training and deployment scalability, underscoring the need for robust infrastructure [40]. The tension is further defined by key dimensions: resource constraints (computational power, personnel, budget) limit both aspects [3]; temporal dependencies create a conflict between prioritizing immediate performance under load (scalability) and ensuring long-term system adaptability and ease of updates (maintainability) [27]; and critical feedback loops

exist where optimizing one dimension negatively affects the other, exemplified by how distributed systems chosen for scalability significantly increase maintenance complexity and introduce new failure modes [16, 29, 42].

The universality of these challenges is evident across diverse domains. In agriculture, for example, processing vast datasets like satellite imagery or IoT sensor data demands scalability, yet frequent model updates needed to adapt to changing environmental conditions require high maintainability [3, 7, 16]. An unresolved maintainability issue here could render a highly scalable system ineffective over time. Similarly, healthcare applications need scalability to handle growing patient data volumes for real-time predictions [4], but also demand maintainability to incorporate new medical findings or comply with evolving regulations [27, 42]; a complex, scalable model might become non-compliant or difficult to update if maintainability is neglected.

While strategic architecture, such as well-designed modularity or microservices, coupled with mature MLOps practices, can potentially mitigate these tradeoffs and improve both dimensions [43], the optimal balance may also evolve, with early projects perhaps favoring maintainability while planning for later refactoring [17]. Despite recognition of this interplay, a significant research gap persists: the literature lacks a comprehensive, systematic analysis that identifies, catalogues, and evaluates maintainability and scalability challenges–and their corresponding solutions–across the distinct stages of the ML workflow [3].

Given these above rationale and motivations, we propose six research questions structured along two dimensions: maintainability and scalability and the stages of ML system development:

- RQ1: What are the data engineering maintainability challenges and solutions in ML systems?
- RQ2: What are the model engineering maintainability challenges and solutions in ML systems?
- RQ3: What are the maintainability challenges and solutions when developing and deploying ML systems in the current ecosystem?
- RQ4: What are the data engineering scalability challenges and solutions in ML systems?
- RQ5: What are the model engineering scalability challenges and solutions in ML systems?
- RQ6: What are the scalability challenges and solutions when developing and deploying ML systems in the current ecosystem?

To structure this analysis, we employ six **research questions** (**RQs**). RQs 1-3 systematically examine maintainability challenges and solutions across these key stages (Data Engineering, Model Engineering, Development/Deployment), while RQs 4–6 similarly address scalability challenges and solutions within the same stages.

This sequential approach facilitates a clear identification of issues specific to each dimension and stage before synthesizing their critical interplay and associated tradeoffs in the discussion section. Given the rapid evolution and integration of ML, this structured analysis aims to provide timely and practical insights for researchers, practitioners, and tool developers, supporting the creation of more reliable, efficient, adaptable, sustainable, and responsible ML systems.

2 Background

2.1 Maintainability of Software Systems and ML

Software engineering defines maintainability as the ease of system modification, updates, or debugging, facilitated by code readability, modularity, and quality documentation. For ML systems, this definition expands significantly due to their data-driven nature and vulnerability to model degradation; maintainability necessitates seamless updates and recalibrations to address shifting data distributions and model changes, ensuring long-term viability [15, 25, 26]. While traditional principles like code quality are relevant [38, 40], ML maintainability uniquely emphasizes

managing evolving data through quality control, lineage tracking, drift handling, and versioning, along with maintaining models via updates, retraining, debugging, version control, and ensuring reproducibility [38, 40].

Furthermore, ML maintainability involves managing complex pipeline workflows, including modifying steps, debugging stages, handling orchestration, and preventing poorly structured "pipeline jungles" [38, 40]. At the system level, considerations include overall architecture, interactions between ML and non-ML components, minimizing "glue code" and "configuration debt," and avoiding entanglement effects like "**Changing Anything Changes Everything**" (**CACE**) [38]. ML systems tend to accumulate technical debt, the long-term consequence of short-term implementation decisions [38, 40]. Consequently, achieving high maintainability requires continuous monitoring for data drift and technical debt, robust version management covering code, data, models, and configurations for effective rollbacks and comparisons [28, 31], and a core MLOps focus on reducing manual effort by managing the interplay of these elements over time, acknowledging that traditional code metrics are inadequate [38, 40].

2.2 Scalability of Software Systems and ML

Scalability refers to a system's capability to manage growing workloads efficiently without compromising performance [5, 6]. While traditional software engineering often addresses user traffic through scaling, caching, and load balancing [6], ML systems present unique, additional dimensions [5]. Beyond user load, ML scalability encompasses Data Scalability for processing and storing expanding datasets, and Model Scalability for training increasingly large models, often necessitating distributed techniques with frameworks like TensorFlow and PyTorch on scalable cloud infrastructure[4, 7].

ML systems must also ensure Throughput/Request Scalability to handle high prediction volumes under strict latency constraints, which involves managing distributed inference resources effectively [4, 7, 14]. Additionally, Resource Scalability, concerning efficient infrastructure use, and Workflow Scalability, scaling the entire ML pipeline, are crucial [5].

Addressing these multifaceted challenges is essential for handling big data, utilizing powerful models, serving numerous users, and optimizing costs, though improvements in one area can sometimes affect others, such as maintainability [5].

3 Related Works

Research on maintainability and scalability challenges and solutions in ML systems, remains relatively limited. Comprehensive studies that systematically catalogue and analyze these issues across the entire ML lifecycle are particularly scarce [40]. This work seeks to address this gap by providing a consolidated view of maintainability and scalability concerns within the context of ML systems.

This SLR builds directly upon our prior work [40], which focused exclusively on identifying maintainability challenges alone. However, we recognized that a more holistic understanding necessitates the inclusion of scalability challenges and solutions, as well as an examination of the often-complex tradeoffs between these two critical aspects. Thus, this study expands the scope by introducing additional RQs that encompass both maintainability and scalability, and navigating these tradeoffs.

To better understand the context of our work, we can categorize existing research into several key areas. Firstly, several studies have explored the broader landscape of challenges encountered during the development and maintenance of ML systems, often without a specific focus on maintainability or scalability. For instance, Nascimento et al. [33] focused on understanding the development process of ML systems, highlighting the general challenges developers face in this rapidly evolving field. Similarly, Nahar et al. [32] investigated collaboration challenges that arise

within teams building ML systems, identifying issues related to communication, coordination, and shared understanding. Serban et al. [39] explored the adoption of software engineering best practices in the context of ML applications, aiming to improve the overall quality and reliability of these systems.

Secondly, some research has focused on the practical aspects of operationalizing and deploying ML models in production environments. Kolltveit and Li [19] conducted a systematic review of techniques, tools, and infrastructures necessary for operationalizing ML models, addressing the gap between model development and deployment. Thirdly, concerns surrounding the reproducibility and replicability of ML research have also been addressed. Liu et al. [23] focused specifically on the reproducibility and replicability of DL studies applied to SE tasks, highlighting challenges and potential solutions in this area.

Fourthly, some studies provided a general overview of software engineering principles as they relate to the development of ML systems. Giray [11] conducted an SLR analyzing the state of SE research for engineering ML systems, providing a broad overview of the field. Wang et al. [44] performed a comprehensive review of SE papers related to ML/DL, covering a wide range of topics and challenges.

Finally, while the aforementioned studies provide valuable context, a few studies have specifically touched upon maintainability and scalability, albeit without an in-depth analysis. Lwakatare et al. [24] conducted an empirical study categorizing SE challenges specific to ML systems, including issues related to maintainability and scalability, offering a useful taxonomy. Zhang et al. [49] discussed the progress, challenges, and research opportunities in building robust ML systems, briefly addressing maintainability and scalability within a broader context. O'Brien et al. [34] explored the concept of technical debt in ML software, identifying various types of technical debt that can indirectly impact maintainability and scalability.

Compared with the existing literature, our study addresses a significant gap by providing a focused and in-depth analysis of both maintainability and scalability challenges, their corresponding solutions, and the crucial tradeoffs between them within ML systems. Unlike our prior work that only reviews maintainability challenges alone [40], this study provides a consolidated catalogue of both scalability and maintainability challenges and solutions, explicitly addressing the interdependent tradeoffs involved in the discussion section. We believe this study will serve as a valuable resource for researchers and practitioners seeking to build robust, maintainable, and scalable ML systems.

4 Research Methodology

This SLR adheres to a rigorous, transparent process to identify, evaluate, and synthesize relevant literature in a reproducible manner [18]. An SLR, as defined by Wohhlin et al. [48], collects and analyzes results from multiple primary studies to provide a comprehensive overview and identify gaps in a specific field. Following established guidelines from Kitchenham [18], this review ensures integrity and high research quality through an evaluable search strategy, providing robust, transferable evidence for further examination.

The SLR process, as depicted in Figure 1, began with a database search using predefined terms and queries to obtain papers addressing the research questions. These papers underwent iterative evaluation based on inclusion/exclusion criteria and quality assessment to ensure the inclusion of only the most relevant and high-quality papers in the review.

4.1 Step 1: Search Strategy

To conduct our SLR, we employed a comprehensive search strategy to identify relevant literature from various databases and digital libraries. The selected databases for our search included IEEE

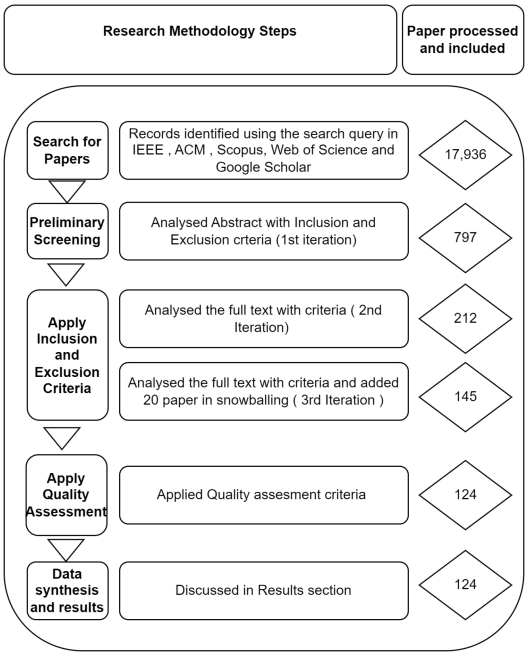

Fig. 1. Systematic literature review process.

Xplore, ACM Digital Library, Web of Science, Google Scholar, and Scopus. These databases were chosen due to their widespread usage and relevance in computer science, software engineering, and ML.

To ensure the identification of the most pertinent literature, we developed a well-crafted search query that encompassed keywords present in the titles, abstracts, and index terms of the papers. The search query utilized a combination of primary keywords derived from our research questions, alternative spellings, synonyms, and additional keywords obtained from pilot searches and related papers. This approach aimed to maximise the coverage of relevant literature across different databases.

The generic query used for our SLR was as follows:

— **TITLE-ABS-KEY** (machine **AND** learning **AND** software **AND** (ml **OR** ai **OR** dl **OR** neural **OR** intelligence **OR** learning) **AND** (adapt* **OR** maintain* **OR** scal* **OR** exten* **OR** evol* **OR** flex*) **AND** (system **OR** architect* **OR** design **OR** build **OR** application **OR** engineering **OR** test) **AND** (data **OR** algorithm **OR** debt **OR** pattern **OR** code))

This search query was designed to capture the essential terms related to our research questions, ensuring that the retrieved papers would address machine learning and associated concepts such as adaptation, maintainability, scalability, evolution, and flexibility. Total number of papers reviewed from each database is shown in Table 1. A more detailed search query used for each database is available in our replication package. Due to platform limitations, our search was restricted to the first 1,000 papers from Google Scholar and the first 2,000 papers from Scopus, sorted by relevance.

To focus our search on recent and relevant publications, we limited the search timeframe to articles published between January 1, 2014, and February 15, 2023. The convergence of factors like the advent of accessible ML libraries (e.g., TensorFlow, PyTorch) [8, 12], GPU advancements, and the rise of deep learning models like AlexNet marked a turning point in 2014. This period saw

Table 1. Number of Papers Reviewed from Each Database

Database	IEEE Xplore	Scopus	Web of Science	ACM	Google Scholar
Papers	4,819	2,000	2,588	1,053	1,000

significant improvements in model maintainability, scalability, and the overall accessibility of ML tools. Consequently, 2014 serves as a suitable starting point for our comprehensive review, as the literature prior to this year is comparatively limited due to the nascent state of the field [8, 12].

We employed a comprehensive search strategy, utilizing a broad range of keywords and their synonyms to ensure exhaustive coverage of the evolving ML terminology [9]. This approach is crucial in capturing the full spectrum of research on maintainability and scalability in ML, given the field's dynamic nature and lack of standardized vocabulary [10, 44]. This approach is crucial, especially in emerging fields, where vocabulary and terminology are still evolving and not uniformly defined this means that authors often use various terms interchangeably or employ related concepts. By including a broad range of keywords and their variations (e.g., "adapt*", "exten*", "evol*", "flex*") and broader aspects of ML systems (e.g., "system," "architect*", "design," "data," "algorithm"), we ensure comprehensive coverage of relevant literature to address our six research questions comprehensively.

While this inclusive strategy may initially introduce some noise, it is essential to avoid overlooking valuable contributions due to terminological variations. To mitigate potential bias, we implemented rigorous quality assessment criteria adapted from Dyba and Dingsøyr [9], which we discuss more in detail in Step 4: **Quality Assessment** (**QA**) in this section, ensuring that our final selection of papers maintains high relevance and quality. This two-step process–a broad initial search followed by stringent filtering–balances the need for comprehensiveness with the focus on high-quality, directly relevant research.

4.2 Step 2: Snowballing

To complement our initial database search and mitigate the risk of missing relevant studies due to evolving terminology, we performed backward and forward snowballing. This step commenced after the second full-text screening iteration, using the set of 145 papers deemed relevant at that stage (see Figure 1) as the starting point.

Backward snowballing involved meticulously examining the reference lists of these 145 articles. Forward snowballing identified subsequent works that cited these articles [48]. This iterative process continued until no new relevant papers were found. All potentially relevant papers identified through snowballing were subjected to the same rigorous inclusion and exclusion criteria (Table 2) via full-text analysis [46].

Through this process, 20 additional relevant research articles that met the inclusion criteria were identified and added to our pool of studies. Snowballing proved effective in uncovering niche works and key studies potentially missed by keyword searches alone, particularly those addressing specific maintainability or scalability challenges and solutions [47]. This brought the total number of papers proceeding to the quality assessment phase to 165 (145 from initial search + 20 from snowballing).

4.3 Step 3: Selection Criteria

To ensure the quality and relevance of the papers included in our SLR, we applied a set of inclusion and exclusion criteria from Table 2 in three iterations, as shown in Figure 1, following the guidelines from Kitchenham [18].

Table 2. Inclusion and Exclusion Criteria

Criteria	Description
Inclusion	
IC1	Data Engineering Challenges in both Maintainability and Scalability Contexts.
IC2	Model Engineering Challenges in both Maintainability and Scalability Contexts.
IC3	Data Engineering Solutions in both Maintainability and Scalability Contexts.
IC4	Model Engineering Solutions in both Maintainability and Scalability Contexts.
IC5	Current Challenges and Solutions when Building an ML System.
Exclusion	
EC1	Non-English papers.
EC2	Publication for which the full text is not available.
EC3	Grey literature (Not Peer reviewed).
EC4	Duplicate papers and shorter versions of already included publications.

Our search queries produced an extensive amount of results, reflecting our aim at prioritising inclusivity and capture as many relevant papers as possible. The challenge of studying maintainability and scalability in ML and SE is compounded by the absence of a standardized and precisely defined set of terms [10]. This challenge becomes particularly prominent in the context of conducting research in these areas. S. Wang et al. [44], in their SLR concerning software engineering and machine learning, also faced a significant number of search results during their search process.

No arXiv content was used in this systematic literature review. The inclusion and exclusion criteria specifically state that only peer-reviewed articles were considered, excluding grey literature (EC3). ArXiv preprints, being non-peer-reviewed, would fall under this exclusion criterion. The review process focused solely on published, peer-reviewed papers that met the specified inclusion criteria.

In the first stage, we analysed the paper's title and abstract to filter out irrelevant papers that did not meet our criteria, as shown in Table 2. In the second and third stages, we conducted a more thorough analysis of the paper's full text to make a final decision on its inclusion or exclusion criteria. It's important to note that no paper that met the selection criteria was included in the quality assessment phase, as depicted in Figure 1.

4.4 Step 4: Quality Assessment

We performed a Quality Assessment (QA) criteria as shown in Table 4 which is adopted from Dyba and Dingsøyr [9]. Wherein two independent reviewers evaluated around 145 papers and performed a Quality Assessment using a Table 4 checklist. In addition to adapting the Quality Checklist, we further enhanced the quality assessment selection by performing this checklist for each Inclusion criterion (IC1 - IC5). We used a 5-point Likert scale for agreement for this assessment, where: 1 - Strongly disagree; 2 - Disagree; 3 - Neither agree nor disagree; 4 - Agree and 5 - Strongly agree. For each given Inclusion criteria (IC1 - IC5). We then normalized and converted this 5-point Likert Scale using a MinMax scaler to obtain a 2-point dichotomous ("yes" or "no") scale adopted by Dyba and Dingsøyr [9]. For a paper to be included, two independent reviewers had to agree on at least one of the Inclusion criteria (IC1 to IC5), which cover at least one of RQ with a dichotomous response of "yes". In case of disagreements, the third reviewer evaluated the record independently, and an agreement based on the majority was reached [10, 18]. We also employed Cohen Kappa inter-rater agreement statistics from this quality assessment response in a dichotomous ("yes" or "no") scale. We got the following agreement score for each inclusion criterion, as shown in Table 3.

Table 3. Inter-Rater Agreement Scores for Inclusion Criteria

Inclusion Criterion	IC1	IC2	IC3	IC4	IC5
Agreement Score	0.7653	0.7853	0.8230	0.7595	0.8388

Table 4. Quality Assessment Criteria [9]

Quality Assessment criteria adapted from Dyba and Dingsøyr [9]
Research Design
QA1. Is the research objective sufficiently articulated and adequately justified?
QA2. Is the context in which the research was conducted clearly stated, including the industry, project setting, utilised products, participants, or observational units?
Data Collection
QA3. Was the data collection carried out comprehensively? For instance, is there a discussion of the data collection procedures and how the research setting might have influenced the collected data?
Data Analysis
QA4. Are the approach and formulation of the analysis effectively conveyed? For example, is there a justification and description of the used analysis method/tool/package?
QA5. Have alternative explanations and confounding factors been considered and discussed in the analysis?
Findings and Conclusion
QA6. Do the findings and conclusions possess credibility? For instance, is the study's methodology sufficiently explained to instil confidence in the findings? Are the findings clearly stated, supported by the data, and consistent with existing knowledge and experience?
QA7. Have the limitations and credibility of the study been adequately discussed?

The kappa statistic ranges from -1 to 1, with 1 indicating perfect agreement, 0 indicating random agreement, and -1 indicating perfect disagreement. Landis and Koch [22] provide guidelines for interpreting kappa values: 0.0 to 0.2 suggests slight agreement, 0.21 to 0.40 suggests fair agreement, 0.41 to 0.60 suggests moderate agreement, 0.61 to 0.80 suggests substantial agreement, and 0.81 to 1.0 suggests almost perfect or perfect agreement. Krippendorff [20] offers a more cautious interpretation, tentative conclusions can be made for values between 0.67 and 0.80, and definite conclusions can be made for values above 0.80. However, researchers often retain kappa coefficients below these cutoffs, as acceptable estimates of interrater reliability depend on study methods and the research question [13]. The agreement in our case is greater the 0.75 for all the Inclusion criteria, which corresponds to the substantial agreement between the raters, as shown in Table 3.

4.5 Step 5: Data Extraction

The codebook in Table 5 was essential for addressing our research questions. It provided a framework to categorize and analyze the challenges and solutions related to maintainable and scalable ML systems, facilitating the identification of key themes and patterns across the entire ML lifecycle.

The maintainable ML codes (addressing RQ1, RQ2, and RQ3) covered a range of data engineering, model engineering, and system-level concerns. For example, codes such as "data cleaning," "data versioning," and "data pipeline" highlighted practices like automated data validation and versioning systems for maintaining data quality and managing data workflows. In model engineering, codes like "model monitoring," "model traceability," and "concept drift" helped identify the importance of robust model monitoring systems and addressing concept drift for maintaining model performance in dynamic environments.

The scalable ML codes (RQ4, RQ5, and RQ6) emphasized the technical and infrastructural challenges of scaling ML systems. Codes like "distributed architecture," "hardware optimization," and

Table 5. Codes for Systematic Literature Review on Scalable and Maintainable ML

Maintainable ML	
Solutions	ML Systems & Apps, Architecture, AutoML, Cloud-based ML, ML System Quality, MLOps, Pipeline jungles, Distributed training, Code quality, Data Engineering, Data cleaning, Data management, Data Pipeline, Data preprocessing, Data validation, Data versioning, Data creation, ML Model Engineering, Concept Drift, Estimator, HPO, Model monitoring, Model deployment, Model traceability, Model publishing and serving, Model training, Model Testing, Multiple Available Solution
Challenges	ML Systems & Apps, Architecture, AutoML, Cloud-based ML, ML System Quality, MLOps, Code quality & smell, Data Engineering, Data cleaning, Data management, Data preprocessing, Data validation, Data versioning, Data creation, ML Model Engineering, Concept Drift, HPO, Model monitoring, Model deployment, Model development, Model publishing and serving, Model traceability, Model training, Model Testing, Multiple Challenges
Scalable ML	
Challenges	Cloud ML, Distributed Architecture, HPO, Edge intelligence, Hardware optimization, Scalable Challenges
Solutions	Data pipeline, Distributed ML, HPC & optimization, HPO, MLOps, Edge computing, Training

"edge computing" helped identify solutions like distributed ML architectures and edge computing for handling large-scale data and computations.

Importantly, the codebook facilitated exploration of the intersection between maintainability and scalability.By systematically applying these codes, we synthesized a comprehensive understanding of the current state of maintainable and scalable ML systems and addressing our research questions.

4.6 Step 6: Data Analysis

After a quality assessment, 124 papers were selected for our research from an initial pool of 145. Each paper was thoroughly examined and assigned one or more RQs based on its content. The papers were then coded using NVivo, a qualitative data analysis software recommended by Kitchenham [18].

In qualitative research, coding is the process of categorizing data using words or phrases that capture the essence of a particular portion of data [41]. In NVivo, coding involves grouping related material into nodes, which represent themes or topics found within the data.

We employed both deductive and inductive coding approaches, starting with RQs as our foundation. This allowed us to extract emergent topics and relationships within the scope of our research questions. The coding process played a crucial role in organizing and categorising the extracted data. Table 5 shows the main codes that emerged for this SLR.

For example, the quotation "Data leakage: a splitting strategy may risk data leakage... This necessitates the need for an iterative trial and error approach to find the best splitting strategy" [LR6] was coded to RQ1, which deals with Data engineering maintainability challenges and solutions under the sub-topic Challenges in "Data preprocessing."

Throughout the analysis, discussions regarding emerging findings ensured code consistency and maintained high levels of abstraction.

4.7 Step 7: Data Synthesis

In the data synthesis phase, we conducted an in-depth analysis to identify key patterns and interdependencies among ML development stages, focusing on maintainability and scalability.

To begin this synthesis step, we examined the data gathered from the literature review and coding process, which included information related to the challenges encountered at each stage of the ML workflow. By analysing the data, we aimed to understand how these stages are interconnected and how they impact the maintainability and scalability of the ML system throughout its development.

Our analysis revealed important challenges and solution for our RQ, For instance, Issues like data leakage can lead to overly optimistic performance estimates and unexpected degradation in production, highlighting the need for robust data pipelines [LR6]. Techniques like distributed training improve scalability but increase system complexity, making debugging and maintenance harder [LR116]. Automation tools improve scalability but can lead to a lack of interpretability and increased difficulty in manual interventions [LR80].

Finally, we distilled our findings into concise tables for each research question, addressing maintainability and scalability challenges separately. These tables serve as a valuable resource for ML tool development and future research.

To ensure transparency and reproducibility, we made all study artefacts publicly available at Zenodo Link.[1] We have also included all the **Literature review** (**LR**) paper with the same prefix **"LR"** for easier navigation and cross referencing in the online supplementary material.

Throughout this process, we adhered closely to the guidelines suggested for conducting SLRs by Kitchenham et al. [18]. We employed a similar approach to the Research design from Edison et al. [10] and Dyba and Dingsøyr [9].

4.8 Threats to Validity

This section outlines four potential challenges to the validity of this systematic review and presents our strategies for addressing each of them. This section is adapted from Edison et al. [10].

4.8.1 Bias in Publication. Publication bias arises when research with positive outcomes is more likely to be published than those with negative results [18, 35]. This may occur when method developers are vested in reporting the method or when organizations studied wish to downplay negative aspects. Reviews like ours are always vulnerable to such bias. In this study, we consider this risk to be significant. To minimise it, we performed a quality assessment and ensured the high quality of the paper using our inclusion and exclusion criteria. We opted not to include grey literature, such as works-in-progress, technical reports, blogs, and unpublished articles.

4.8.2 Identifying Primary Studies. In order to minimise the risk of overlooking or excluding relevant papers, we aimed to gather as many peer-reviewed articles as possible related to the maintainability and scalability of the ML system and application. We also employed a snowballing search during the full-text review by identifying relevant papers in the reference lists of each article [21]. It is impossible to entirely eliminate the risk of missing relevant articles. Inconsistent or varying terminology in ML research may have influenced identifying primary studies [37].

4.8.3 Consistency in Selecting Primary Studies and Data Extraction. The review process began with the establishment of inclusion and exclusion criteria, ensuring a consistent framework for assessing studies. Initially, we focused on the titles and abstracts of the studies (Figure 1). However, a challenge arose during pilot reviews due to the lack of explicit definitions for "Maintainability" and "Scalability" in some studies, leading to ambiguity. To address this, we established clear definitions and contexts for both terms, which guided the second iteration of the review involving a detailed examination of the full texts of the studies.

In the final iteration, approximately 145 papers selected after initial screening underwent a thorough quality assessment by two independent reviewers. The aim was to ensure agreement among reviewers regarding study quality, as shown in Table 3. A meeting was convened to assess the review checklist and resolve any differences in assessments, resulting in a comprehensive and unanimous decision-making process.

[1] https://zenodo.org/doi/10.5281/zenodo.8024833

Table 6. Data Engineering Maintainability Challenges

Challenge	Description of the challenges	ML stage	Paper
CM01	Creating and labeling datasets	Dataset Creation	LR1,2,9,15,76
CM02	Data susceptibility to error, drift, bias, attacks	Dataset Creation	LR1,2,17,71,89
CM03	Poor documentation and transparency	Dataset Creation	LR16
CM04	Missing data	Data Preprocessing	LR1,6
CM05	Order of data processing steps	Data preprocessing	LR1,2
CM06	Data imbalance and leakage	Data preprocessing	LR5,6
CM07	Collaboration difficulties due to limited tools	Data preprocessing	LR6,8
CM08	Maintaining reliable validation pipeline	Data validation	LR2,12
CM09	Data quality and validation infrastructure	Data validation	LR13
CM10	Evolving ML datasets and metadata assets.	Data Management	LR 3, 4,14,15, 68
CM11	Missing tools for reproducibility, traceability, and auditability	Data Management	LR14,15,68

4.8.4 Reliability of Extracted Data. We discovered that some studies needed more precise information regarding a clear connection between SE and ML maintainability. In these instances, we had to rely on the researcher's judgment. For example, some papers could have specified clearly how different or unique ML maintainability and scalability are different from the SE perspective. In cases where the method was not explicitly stated, we used the study context and practices reported in related studies to classify the paper. Consequently, some of the extracted findings may be partially inaccurate. To mitigate this, each reviewer refined and verified the extracted data during the Quality Assessment stage.

5 Results

This section offers a comprehensive summary of the maintainability and scalability challenges and solutions encountered in ML workflows, drawing from a **systematic literature review** (**SLR**). These challenges and solutions have been organized using NVIVO software and categorized according to the different stages of the ML workflow, in alignment with specific RQs. The relationship between these challenges and their respective solutions is elucidated through a coding system: CM01 to CM41 for maintainability challenges and CS01 to CS13 for scalability challenges.

Each maintainability challenge is linked to its corresponding solution, facilitating a clear understanding of how to address such issues (Tables 6 and 7). A similar approach is used for scalability challenges, where each challenge is paired with its solution (Tables 12 and 13).

For a comprehensive overview of the references used in our LR, a complete list is available as Online Material. Each reference has been assigned a unique identifier for easy navigation and cross-referencing. This streamlined approach ensures quick and accurate retrieval of information, enhancing the overall accessibility and usability of the research.

5.1 RQ 1. Data Engineering Maintainability Challenges and Solutions

This section focuses on our research into Data Engineering Maintainability Challenges and Solutions, specifically addressing RQ1. We explore critical challenges in ML workflows, each labeled with a unique code like CM01, and propose corresponding solutions, such as SM01.

The research findings are systematically summarized in two key tables. Table 6, 'Data Engineering Maintainability Challenges,' lists challenges in ML workflows, such as 'CM01' (maintainability challenges), and correlates them with relevant stages and literature sources. Each challenge is linked to specific references from the LR papers, like 'LR09' and 'LR15' (e.g., CM01 [LR9,15]), which discuss these challenges in detail. These papers, which delve into these challenges in greater detail,

Table 7. Data Engineering Maintainability Solution for the Identified Challenges in Table 6

Index	Solutions	Solution/Tools/frameworks/methods
CM01	SM01	Crowdsourcing, AWS Mechanical Turk, Weak Supervision [LR2,3], Semi-Supervised Learning and Data programming, Shuffler [LR3], Inspector gadget [LR9], Gamification [LR15], OneLabeler[LR76]
CM02	SM02	Worst ofK, Sensei [LR17], AugMIX [LR89, DeepCloak methods [LR40]
CM03	SM03	Guidelines from [LR16]
CM04	SM04	KNNI, WkNNI, KMI, IRMI [LR5] for Missing data (imputation).
CM05	SM05	MLClean [LR1]
CM06	SM06	SMOTE , ROSE [LR5], Random , Baseline and time based splitting [LR6]
CM07	SM07	Ballet [LR7], Cylon [LR8] , Domino Data lab , Dataiku [LR7].
CM08	SM08	Tensorflow validation, Facets in TFX, SEEDB, CUDE [LR2], Data Linter, Deequ [LR13]
CM09	SM09	Tensorflow validation in TFX [LR2,13]
CM10	SM10	Shuffler[LR3], JUNEAU [LR4]
CM11	SM11	DVC, MLFlow, Pachyderm, Quilt Data[LR14], NeptuneML, KubeFlow, WandB, ModelDB [LR68]

are conveniently accessible as Online Material and also on Zenodo, ensuring easy access for the reader.

Following this, Table 7, "Data Engineering Maintainability Solutions for the Identified Challenges," pairs each challenge (e.g., CM01) with a corresponding solution (e.g., SM01), presenting a range of tools and methods, each backed by relevant literature references. This layout offers a concise and comprehensive guide to addressing data engineering maintainability in ML workflows, supported by easily accessible in-depth literature for interested readers.

5.1.1 Dataset Creation Challenges. Collecting and preparing datasets for ML/DL research or applications can be challenging and time-consuming. Data gathered from external or evolving sources may be susceptible to errors, missing values, outliers, and even malicious attacks. Inherent biases can also creep into the data, impacting the fairness and accuracy of the resulting models (**CM01, CM02** [LR1,2]).

Creating labels for training data often requires the help of humans, but the repetitive nature of this task can quickly become tedious and demotivating for the annotator (**CM01** [LR9,15]). Furthermore, developing custom labelling tools for different applications is challenging due to the need for cross-disciplinary knowledge, and existing labelling tools are often difficult to customise (**CM01** [LR76]).

ML models can misclassify data due to adversarial attacks and variations in a production environment, making it challenging to obtain a mature model that generalizes well to real-world scenarios (**CM02** [LR17,71]).

Existing data augmentation techniques aimed at improving robustness against corrupted data or adversarial attacks often come with significant tradeoffs in terms of accuracy. Probabilistic and Bayesian neural networks can struggle under data shifts when using these techniques. There's also the risk that aggressive data augmentation schemes may improve clean accuracy at the cost of robustness against corruption or adversarial attacks (**CM02** [LR89]).

Finally, datasets used in ML are often poorly documented, poorly maintained, and lack transparency in their creation process, contributing to errors and perpetuating bias (**CM03** [LR16]).

5.1.2 Dataset Creation Solution. The creation of high-quality labels for training data is crucial for ML models. Manual labeling is often expensive and time-consuming, so alternative strategies are implemented, such as crowdsourcing via Amazon Mechanical Turk, weak supervision, semi-supervised learning, and data programming (**SM01** [LR2,3,9]). These techniques use human knowledge and other tools to facilitate the labeling process, with data programming being particularly successful in various applications. Tools like Shuffler and Inspector Gadget aid in managing

annotations and weak labels, respectively (**SM01** [LR3,9]). Zhang et al.'s OneLabeler implements a flexible, customizable design for labeling tools, validated by extensive studies (**SM01** [LR76]). Gamification is also employed to enhance user engagement in data annotation (**SM01** [LR15]).

Further focus on the robustness of machine learning models involves the use of data augmentation techniques such as the "Worst ofk" method and Augmix, which has shown significant improvements in challenging benchmarks (**SM02**[LR17,89]). In an adversarial setting, Wang et al. present a multi-faceted defense strategy aimed at enhancing the robustness and privacy of the ML model using techniques like differential privacy and homomorphic encryption (**SM02**[LR40]).

Lastly, responsible AI use mandates an emphasis on transparency and accountability, incorporating practices like treating datasets critically, conducting peer reviews, using visualization tools, and understanding dataset limitations to ensure ethical deployment and usage (**SM03** [LR16]).

5.1.3 Data Pre-Processing Challenges. Maintaining the quality of data in the preprocessing phase involves challenges related to data errors, missing values, outliers, and bias. This can affect the accuracy and fairness of the model, necessitating precise data handling (**CM04**[LR1,6]). The order in which data preprocessing steps are carried out, particularly data cleaning, sanitization, and unfairness mitigation, indirectly influences model accuracy (**CM05**[LR1,2]). However, there is a notable lack of robust empirical evidence to support this observation. This gap suggests a need for further research to empirically validate how the sequencing of these preprocessing steps impacts the overall accuracy of machine learning models. Other issues include handling imbalanced data and data leakage between training and validation sets, both of which impact the maintainability of datasets (**CM06**[LR5,6]).

In the realm of open data science, there are hurdles in dividing and combining tasks and code among multiple contributors. The current tools may not integrate well with various languages, reducing productivity.

The Big Data frameworks such as Apache Hadoop, Twister, Apache Spark, and Apache Flink are unable to satisfy the escalating demands for more efficient and integrable frameworks, especially considering the needs of Machine Learning (**CM07**[LR6,8]). Even attempts to match user-friendly frameworks like NumPy and Python Pandas result in compromised performance due to runtime overheads (**CM07**[LR8]).

5.1.4 Data Pre-Processing Solution. Missing values in datasets can be handled by discarding or imputing values. K-nearest neighbor algorithm (KNN) and its variations, KNN imputation (KNNI) and weighted KNNI (WKNNI), are standard methods for imputation, alongside K-means Clustering Imputation (KMI) (**SM04** [LR5]). Karanikola et al. [LR5] introduced an improved Iterative Robust Model-based Imputation (IRMI) combining boosting and decision tree theory, which outperforms other IRMI methods (**SM04** [LR5]).

In data processing, the order of steps, such as cleaning, sanitization, and unfairness mitigation, impacts the model's accuracy and fairness. Tae et al. [LR1] found that cleaning and sanitization should be done together, followed by unfairness mitigation, and developed a framework called MLClean for enhanced accuracy, fairness, and runtime (**SM05** [LR1]).

Imbalanced data affects modelling, and techniques such as over-sampling, under-sampling, SMOTE, and ROSE can balance class distribution (**SM06** [LR5]). Mitigating data leakage involves alternative splitting methods such as Random, Time-based, and Baseline splitting (**SM06** [LR6]).

Ballet provides a lightweight solution for integrating features in data science projects and enhances collaborative model development, unlike commercial platforms like Domino Data Lab and Dataiku, which lack specific collaboration and incremental model development features (**SM07** [LR7]). Cylon, a data engineering toolkit, integrates ML with other data processing systems, aiming

for a more robust and efficient data engineering pipeline and increased user productivity through seamless integration with existing frameworks (**SM07** [LR8]).

5.1.5 Data Validation Challenges. Ensuring a reliable ML validation pipeline is complex due to challenges like schema problems, data changes, drift, and vulnerabilities to attacks and errors (**CM08** [LR2]). Malicious data poisoning further complicates the situation. Traditional data validation tools like TensorFlow validation are designed to create schemas for validating future datasets. However, they often face challenges in dealing with data drift and unreliable data sources. This limitation can adversely affect the accuracy of machine learning models. Another significant challenge lies in the interpretability of ML models, for instance, assessing their effectiveness can be difficult without well-defined test cases. Furthermore, continuous updates and changes in both the models and data features can result in models that reflect outdated information rather than current realities. (**CM08** [LR12]).

Integrating data validation tools into developing ML systems is vital but requires substantial engineering resources. Challenges stemming from the stochastic nature of ML models, data drifts, and dependencies between features can hinder their validation and monitoring, leading many teams to overlook data validation workflow because of its complexity (**CM09** [LR13]).

5.1.6 Data Validation Solution. Machine learning platforms, such as **TensorFlow Extended** (**TFX**), have practical data validation tools that can find and resolve errors in data using methods such as data visualization and schema generation. For instance, Facets in TFX offer vital data statistics for machine learning, while tools like SeeDB and CUDE apply hypothesis testing to verify the significance of results. This data validation process is crucial for spotting problems in the data that could affect the machine learning pipeline. Also, TensorFlow Data Validation simplifies the process of creating database schemas from previous datasets and validating future datasets with the schema, improving the quality and dependability of training data (**SM08, SM09** [LR2]).

Additionally, tools like Data Linter, Deequ, and TensorFlow Data Validation are used to inspect and suggest ways to improve training data representation automatically. Best practices for adopting a data validation process and tool for machine learning projects include defining data quality tests, providing actionable feedback, and treating data errors with the same severity as code. This approach can minimise manual effort, identify data errors early, and provide a testing approach for machine learning-enabled software systems (**SM08, SM09** [LR13]).

5.1.7 Data Management Challenges. Handling expansive and continuously evolving datasets in machine learning projects requires significant infrastructure. This infrastructure must be capable of managing a variety of data-related tasks, including feature extraction, data acquisition, and serialization. There is an increasing need for efficient tools that can adeptly handle processes like data acquisition, integration, modification, serialization, and storage across different formats (**CM10** [LR3]). Additionally, the iterative nature of changes during the exploratory phases of ML projects often necessitates manual adjustments. This can make the transition from exploratory data analysis to a more regulated, reproducible workflow quite complex. Such complexities can lead to issues with versioning and maintainability, making these processes prone to errors (**CM10** [LR4]).

The significance of tracking data at each stage for reproducibility cannot be handled efficiently by traditional version control systems. Specialized tools are required for managing complex workflows and ensuring aspects such as traceability, auditability, and reproducibility (**CM10, CM11** [LR14,15, 68]).

Enhancing the model development process requires a more structured approach to managing ML assets, including improved collaboration tools. These must address collaboration hurdles among developers and provide utilities for comparison, analysis, and interpretation of results,

facilitating deeper insights into ML experiments (**CM10** [LR68]). The overarching challenges of infrastructure requirements, lack of proficient tools, data complexity, continuous changes, and the absence of collaboration between teams, present a comprehensive issue in the smooth functioning and maintainability of ML projects.

5.1.8 Data Management Solution. The increasing necessity for publicly accessible tools for data management in tasks like data acquisition, labeling, feature engineering, and serialization in various formats is evident. Tools like Shuffler are designed for annotating and labeling images, utilizing relational databases and SQL queries, and supporting image manipulation, object detection, semantic segmentation, and object matching tasks (**SM10** [LR3]). Similarly, JUNEAU assists in managing tabular data by transforming Jupyter Notebooks' backends and user interface, functioning as a data lake management subsystem that captures and indexes external files and computational steps within notebooks (**SM10, SM11** [LR4]).

ML experiment management tools like MLFlow, NeptuneML, WandB, DVC, Pachyderm, ModelDB, and Quilt Data contribute to enhancing ML pipelines' efficiency by minimizing manual intervention, supporting reproducibility, and traceability. They employ infrastructure-as-code principles to manage data and model pipelines and experiments in open-source projects **SM11**[LR14]. Experiment management tools, such as MLFlow, Neptune, KubeFlow, and ModelDB, offer functionalities to store, track, and version assets across different experimental runs. A subset of these tools focuses explicitly on deep learning support (**SM10,11** [LR68]). Together, these tools represent significant advancements in ML data management and pipeline efficiency, reflecting the evolving needs of the field.

5.2 RQ2. Model Engineering Maintainability Challenges and Solutions

This section addresses RQ2, exploring Model Engineering Maintainability Challenges and Solutions. Table 8 identifies challenges (e.g., CM12) within ML model engineering, linking them to specific ML stages and relevant literature. Table 9 connects each challenge (e.g., CM12) to potential solutions (e.g., SM12), offering tools, frameworks, and methods, along with supporting literature.

5.2.1 Model and Concept Drift Challenges. Maintainability challenges in ML can stem from concept drift, or changes in data distribution and variable relationships over time, causing a loss of accuracy in algorithms (**CM12** [LR6,18,20]). Various methods, such as supervised, semi-supervised, unsupervised, statistical, and evolutionary algorithms, have been developed to handle concept drift, but none are universally suitable. Machine learning ensembles have been proposed but may affect detection speed due to retraining and selection strategies (**CM13** [LR19]). Additionally, dataset shifts, such as covariate shift, prior probability shift, and concept shift, can cause a drop in system utility and accuracy (**CM12** [LR77]). Data distribution-based approaches can track changes but may be limited in detecting certain types of drift or addressing drift on individual models or paired streams (**CM12** [LR79]). Despite being a long-standing problem, concept drift detection remains an intricate and costly process, often leading to model obsolescence and performance degradation (**CM14** [LR19,20]).

5.2.2 Model Drift Solution. Concept drift refers to the alteration in data distribution over time, negatively impacting the precision of machine learning models. There are various strategies to counteract concept drift, such as statistical testing, online learning, and the Streaming Ensemble Algorithm. Nishida et al. [LR6] suggested incremental updates to models instead of complete retraining. Streaming Ensemble Algorithm and Kappa Updated Ensemble are examples, employing majority-voting and dynamic weighing (**SM12** [LR6]). The ADWIN drift detector, using a

Table 8. Model Engineering Maintainability Challenges

Challenge	Description of the challenges	ML stage	Paper
CM12	Models becoming inaccurate due to data distribution changes	Model drift	LR6,18,20,77,79
CM13	Choosing the right drift detection algorithm.	Model drift	LR19
CM14	Maintaining drift detection pipeline.	Model drift	LR19, 20
CM15	Time-consuming and computationally intensive	HPO	LR22,23,24
CM16	Determining the right HPO approach.	HPO	LR21,22,24
CM17	Monitoring ML models in production.	Monitoring	LR25,20,27
CM18	Challenges in standardisation, metrics selection, feedback loops, and model drift.	Monitoring	LR26,27,28,77
CM19	Deploying ML models in real-world settings	Deployment	LR11,20,28
CM20	Many different types of hardware and software platforms.	Deployment	LR29,30,27
CM21	Maintaining reproducibility and updating software/hardware.	Deployment	LR72,28
CM22	Ensuring ethics, compliance, robustness, and generalization	Governance	LR20,88, 73
CM23	Reproducibility and tracking dependencies and artefacts.	Governance	LR33,34,71
CM24	Managing risks associated with the model.	Governance	LR73,88,71
CM25	Maintaining and replicating results of model training.	Model training	LR32, 35
CM26	Choosing between retraining and incremental modelling.	Model training	LR18
CM27	Managing and integrating training pipelines.	Model training	LR18,20,27
CM28	Challenges in model testing and maintenance.	Model Testing	LR36,40,41,43,71,86
CM29	Oracle problem in ML testing	Model Testing	LR39,86
CM30	Testing ML algorithms due to their stochastic nature.	Model Testing	LR 38, 78, 86
CM31	Collecting and ensuring test data quality.	Model Testing	LR39,86,83,71

Table 9. Model Engineering Maintainability Solution for the Identified Challenges in Table 8

Index	Solutions	Solution/Tools/frameworks/methods
CM12	SM12	Statistical testing, online learning, and the Streaming Ensemble Algorithm. Incremental learning rather than retraining [LR6]. ADWIN drift detector [LR18], Driftage [LR19], Quad [LR79]
CM13	SM13	No clear options, claimed methods Online sequential extreme learning machines (OS-ELMs) [LR90]
CM14	SM14	ADWIN drift detector[LR18], Driftage framework [LR19], QUAD[LR79].
CM15	SM15	Google Vizier, Amazon SageMaker, and SigOpt.Optunity and Tune , Auptimizer [LR21] . HyperNOMAD [LR22]. Hyperopt, Auptimizer, Katib, and Tune [LR23,24]. Ultron-AutoML [LR24]
CM16	SM16	Bayesian approaches, evolutionary algorithms, multi-armed bandits, Reinforcement Learning, and Neural architecture search[LR21]
CM17	SM17	Overton [LR25], WandDB, MLflow, NeptuneML, ModelDB, KubeFLow[LR68]
CM18	SM18	Online Testing [LR27] , Embracing MLOps [LR28,85]
CM19	SM19	Infrastructure maintenance and collaboration in teams [LR20] , Automation, MLOps [LR28 , 72]
CM20	SM20	TF Lite and Core ML[LR29] ,
CM21	SM21	DLHub[LR33], MLC[LR72], KFserver, Polyaxon, and Kubeflow [LR72].
CM22	SM22	EU AI HLEG trustworthy AI[LR73] , Governing AI system through Code-enforced rules [LR88]
CM23	SM23	DLHUB [LR33], MSR4ML framework [LR34], MLFlow, NeptuneML, and WandB, KubeFlow, and ModelDB [LR68], Track and store relevant model information and metadata[LR85]
CM24	SM24	EU AI HLEG[LR73], Model verification involving requirement encoding, formal verification, and test-based verification [LR20]
CM25	SM25	Use Simpler model [LR20], Local learning[LR35] MLFlow, NeptuneML, and WandB, KubeFlow, and ModelDB [LR68]
CM26	SM26	Homogeneous and heterogeneous Transfer Learning [LR69]
CM27	SM27	Neptune, KubeFlow, and ModelDB [LR68]
CM28	SM28	EVOSUITE and Randoop [LR36], Speller [LR37], DeepXplore, Themis [LR39] , dpEmu [LR41], TensorFI [LR42] , CALLISTO [LR44] , Ares, TensorFI, and PyTorchFI [LR71] , TensorFlow Data Mutator (TF-DM) [LR71]
CM29	SM29	Potential solution and recommendation [LR86]
CM30	SM30	XSEED [LR78]
CM31	SM31	No good solution found in this SLR

Relative Standard Deviation (**RSD**) metric, is commonly used to gauge drift in data streams (**SM14, SM12** [LR18]). MAPE-K (Monitor-Analyse-Plan-Execute over a shared Knowledge) pattern, implemented in the Driftage framework, is applied for drift detection in multi-agent systems using Python (**SM14, SM12** [LR19]).

To mitigate the impact of data shifts, various tactics like component replacement, human-based labeling, transfer learning, and retraining/hyperparameter optimization can be employed, depending on context and problem (**SM12** [LR77]). The QuaD method offers improvements in concept drift detection across multiple models using feature stores, SHAP values, and Collaborative Filtering (**SM12, SM14** [LR79]).

Yang et al. [LR90] categorize approaches to adapting to concept drift into passive and active. Passive methods periodically retrain models, while active ones continuously monitor and adapt them. A new method using online sequential extreme learning machines (OS-ELMs) is proposed, allowing continuous updates and outperforming existing state-of-the-art drift detection techniques (**SM13** [LR90]).

5.2.3 HPO Challenges. Maintainability challenges in ML models include difficulties tuning models and hyperparameters, which can be time-consuming and require much computational effort. **Hyperparameter Optimization** (**HPO**) is a critical aspect of ML and can be challenging due to the high dimensionality of the search space and the need to balance exploration and exploitation. Various approaches such as Random Search, Grid Search, Bayesian Optimization, and Gradient-based methods have been proposed to solve the HPO problem, but each of them has its own limitations (**CM15, CM16** [LR22, 24]). Automated solutions for hyperparameter tuning are available but come with challenges, such as the lack of a standard API among current open-source solutions. Different HPO algorithms have different strengths, such as working well for low-dimensional numerical problems or high-dimensional structured model spaces (**CM16** [LR21]). Without expert knowledge, it is often done through trial and error, which is very time-consuming and resource intensive. HPO impacts models' performance, efficiency, and convergence rate, and wrong choices can affect learnability and training progress (**CM15** [LR23, 24]). Moreover, it requires setting up and maintaining an orchestration pipeline to run the optimization and keep track of the parameters and results for reproducibility (**CM15** [LR21, 22]).

5.2.4 HPO Solution. Researchers have proposed various methods for tuning machine learning models, such as Bayesian approaches, evolutionary algorithms, multi-armed bandits, and Neural architecture search by learning, to speed up the process. Efforts have been made to automate the process, such as Google Vizier, Amazon SageMaker, and SigOpt. However, these do not allow for customization or easy integration with other resources. Open-source projects like Optunity and Tune offer user-friendly APIs but also have limitations in terms of customization and integration (**SM15, SM16** [LR21]). Auptimizer is an open-source automated optimizer framework that addresses the challenges of using HPO in practice by reducing the effort to use and switch HPO algorithms, providing scalability for cloud/on-premise resources, simplifying the process to integrate new HPO algorithms and new resource schedulers, and tracking results for reproducibility (**SM15** [LR21]). HyperNOMAD package is a tool that uses the NOMAD algorithm to tune the hyperparameters of **deep neural networks** (**DNNs**). Cloud-based services like AWS Sagemaker, Azure ML, and Google Cloud AI platform offer hyperparameter tuning. However, users can only choose from the algorithms provided and have limited options for customization (**SM15** [LR22]). Open-source frameworks like Hyperopt, Auptimizer, Katib, and Tune offer more flexibility but may not be designed for reliable and efficient use of resources (**SM15** [LR23,24]). Ultron-AutoML is a service that can be installed on a Kubernetes cluster or a cloud platform like AWS, Google Cloud, or Azure ML. The framework is designed with reliability, efficiency, and scalability in mind to ensure the completion of a user's HPO job in a timely and cost-effective manner. Additionally, it allows for ease of use, flexibility, and customization for the user (**SM15** [LR24]).

5.2.5 Model Training Challenges. Training a model can be challenging due to its computational demands and difficulty validating and replicating results. Additionally, deep learning may not be the best approach for all types of problems, and it can be computationally expensive and challenging to fine-tune (**CM25** [LR35]).

Retrained and incremental modeling are two standard methods for training ML models, but evidence to justify the choice between these methods is often lacking. Choosing between retrained and incremental modeling does not affect model interpretation, but it does affect learning assumptions and algorithm variants, leading to different accuracy and training time tradeoffs (**CM26** [LR18]).

Training models can also be costly, particularly in the deep neural network, and concerns have been raised about the environmental impact of training ML models. Additionally, training performance at runtime with evolving data streams is an even more significant maintainability challenge because the need for constant retraining with new data adds to the complexity of managing, integrating, and deploying the training pipeline to other systems and applications (**CM27** [LR18, 20]). Furthermore, Model training also poses maintainability challenges, including setting up the infrastructure to automate the pipeline and monitor performance (**CM25** [LR32]).

When training in on-premises environments, practitioners are often limited by the restricted usage of libraries and software packages and only allow the use of audited tools and platforms within the organization for sensitive data. All dependencies must be approved before use and are stored in a private repository with only allowed packages that have been internally audited. The challenges in modeling include (1) limitations in using the latest ML technologies, (2) the importance of baseline models, (3) tracking all experiments, which often require a custom spreadsheet, and (4) defining problem-specific performance metrics, hindering the definition of standards (**CM27** [LR27]).

5.2.6 Model Training Solution. In improving **Deep Learning** (**DL**) model performance, transfer learning emerges as a compelling method, allowing the reuse of existing models instead of building them from scratch, thus saving time and computational resources (**SM26** [LR2]). Two categories exist within transfer learning: homogeneous transfer learning assumes the same feature space but different data distributions, while heterogeneous transfer learning assumes different feature spaces (**SM26** [LR69]).

DL models may not always be the most suitable option. Simpler models such as shallow networks, decision trees, and random forests can act as effective starting points for machine learning solutions (**SM25** [LR20]). The concept of local learning, as demonstrated by Majumder et al., can further enhance performance by training specific models for particular data regions (**SM25** [LR35]).

Chen et al. [LR18] provided insight into two key training methods: retrained modeling, where new models are trained from scratch, and incremental modeling, where existing models are updated. The study concluded that the choice between the two depends on the learning algorithm's characteristics, with incremental modeling demonstrating better accuracy on Bagging and less error on Boosting, and an overall tradeoff between accuracy and training time (**SM26** [LR18]).

Lastly, the landscape of machine learning experiment management tools such as MLFlow, NeptuneML, WandB, KubeFlow, and ModelDB still have room for growth and improvement. Despite their capabilities in handling reproducibility and traceability, issues like a lack of interoperability and coupling with specific libraries are current challenges. These necessitate further assessment and development in tooling and engineering processes (**SM25, SM27** [LR68]).

5.2.7 ML Testing Challenges. ML systems pose unique challenges in software testing and maintenance, attributable to their complex, evolving nature and unpredictable behavior. Traditional testing methods are often inadequate for ML systems due to their vast input spaces and lack of

statistical guarantees for out-of-distribution data (**CM28** [LR32], **CM30** [LR38]). These challenges include the difficulties in deploying and maintaining ML systems, offline and online testing, and the constant evolution that makes conventional test oracles obsolete (**CM29, CM31** [LR39]).

A deeper inspection reveals problems in understanding ML system behavior due to the absence of specifications or source code, challenges with data pipelines and redundant experimental code paths, and complexities in testing for valid behaviors (**CM28** [LR32, 36, 40]). Moreover, identifying genuine bugs within ML systems can be difficult as individual test failures may not always indicate an issue.

The vast input space of **ML systems** (MLSs) makes the revealing of data-sensitive faults tough, necessitating ML-specific testing criteria like mutation adequacy. Various levels of MLS testing occur, from input testing to system testing, with challenges in generating realistic input data and scalability assessment (**CM28, CM29, CM30, CM31** [LR86]).

The emulation of data faults to assess their impact on ML systems is another hurdle. Although frameworks for handling data faults exist, the consequences of faults in the ML pipeline can be severe, especially in critical systems such as autonomous vehicles. Current techniques and tools for evaluating ML models with data faults are limited and may include adversarial attacks or noisy data (**CM28, CM31** [LR41, 43, 71]).

Additional challenges involve ensuring data quality for high-dimensional data types like natural language, labor-intensive test data collection, efficient test execution in complex systems with entangled ML components, and differentiating between genuine test failures and test flakiness due to the inherent randomness in ML algorithms (**CM31** [LR83], **CM30** [LR78]).

5.2.8 ML Testing Solution. The empirical analysis of ML libraries has revealed several areas of concern and avenues for future research in ML testing solutions. A study was conducted on five popular ML libraries using tools such as EVOSUITE and Randoop, finding that most of them lack adequate unit test suites, with quality metrics such as code coverage and mutation score falling short (**SM28** [LR36]). Tools like DeepXplore and Themis were also explored to identify bugs and discrimination in ML systems (**SM28** [LR39]).

Zhu et al. proposed a functional regression testing method for an ML spelling checker/corrector, leveraging data perturbation and clustering to enhance testing efficacy (**SM28** [LR37]). Berend et al. evaluated the effects of **deep learning** (**DL**) systems with adversarial retraining, highlighting the importance of data distribution awareness, and improving retraining by up to 21.5% [LR38]. Pastor et al. provided an overview of current testing practices, underscoring the importance of a multifaceted approach [LR32].

The prototype framework dpEmu facilitates the modeling of data faults in ML systems, aiding in the analysis of accuracy and robustness tradeoffs (**SM28** [LR41]). TensorFI and CALLISTO are frameworks for evaluating the resilience of TensorFlow-based ML applications and addressing training data quality, respectively (**SM28** [LR42, LR44]). Panichella et al. elaborated on mutation testing, focusing on DL, and suggested future research [LR45].

Dutta et al. conducted a study on seed usage in ML project testing, offering insights into seed usage and recommending alternative strategies with XSEED (**SM30** [LR78]). The stochastic nature of testing ML systems poses challenges, calling for solutions like automated oracles, common frameworks, and diverse test data generation methods (**SM29** [LR86]). The introduction of fault injection techniques has fostered the development of tools like Ares, TensorFI, PyTorchFI, and TF-DM, targeting various data faults and bolstering robustness in ML applications, especially in safety-critical systems (**SM28** [LR71]).

5.2.9 Model Monitoring. Maintaining and improving ML models in production is a high cost, and engineers face challenges such as monitoring fine-grained quality and diagnosing errors in

ML models, applications, and data pipelines. There is limited tooling for model lifecycle support, and engineers need tools to improve and maintain quality in the face of changes to input distribution and new production features (**CM17** [LR25]). Additionally, using system logs to monitor the behavior of deployed models is a crucial issue in maintaining ML systems, as the community is still in the early stages of understanding the key metrics and methods for monitoring data and models (**CM18** [LR26]). Each model requires a unique approach, and metrics teams use self-developed or highly customized dashboard platforms to monitor the models. However, standardization is lacking, which may lead to a bug-prone, ad-hoc, and non-reusable measurement system. Automation is needed for model monitoring; therefore, teams have created tools that are not easily shared across the organization and are difficult to maintain (**CM17** [LR20,27]).

No standard approach or metrics exist for monitoring each model. Furthermore, a change in the distribution of data points between the training and testing environments can decrease accuracy and ultimately reduce system utility (**CM18** [LR77]) which may be difficult to notice without proper monitoring tools. Monitoring machine learning models is a complex task that requires an ongoing understanding of metrics and methods. There are challenges, such as feedback loops, where models can influence their behavior over time. Automation and standardization are needed to improve monitoring, as well as address issues like prediction bias (**CM18** [LR 27,28]).

5.2.10 Model Monitoring Solution. Monitoring the behavior of machine learning models during deployment is vital to ensure proper functioning through a practice called Online testing (**SM18** [LR27]). This need arises due to potential challenges like degradation from non-stationary data distribution or hardware issues. A two-stage process has been designed for monitoring staleness and updating the model to enhance its quality. The monitoring stage assesses the model's performance against success criteria, followed by an update decision based on cost-benefit analysis (**SM18** [LR85]). Techniques for monitoring include semi-automated methods using MLOps, with features such as triggers for degradation and tools for drift. Transitioning towards fully automated monitoring includes integration of various aspects like CI/CD, CT pipeline, model certification, security, explainability, auditing, reproducible workflows, and end-to-end quality tests (**SM18** [LR28]).

Additionally, managing log data in a DevOps environment requires introducing new processes, and this can support anomaly investigation by linking machine-understandable entry with human-readable logs (**SM17** [LR26]). Tools like Overton allow for streamlined model creation, deployment, and monitoring without code writing (**SM17** [LR25]). Experiment management tools, such as MLFlow, NeptuneML, and WandB, handle aspects like reproducibility and traceability but may not be sufficient. Therefore, an evaluation and development of new tools and engineering processes is essential to comprehensively manage machine learning models in production (**SM17** [LR68]).

5.2.11 Model Deployment Challenges. Deploying Machine Learning models in real-world environments presents a series of complex challenges that are critical to both deployment and maintenance. These include managing dependencies, maintaining glue code, monitoring, logging, and handling unintended feedback loops (**CM19** [LR11,20]). The difficulties are further pronounced in the migration and quantization processes, where understanding their impacts on prediction accuracy and performance becomes vital. Some issues related to maintainability, such as keeping software and hardware up to date and maintaining reproducible results, often lead to increased engineering and human resource costs (**CM19** [LR28]).

The release and maintenance of ML models involve multiple intricate steps like packaging, validation, deployment to production, and continuous updating and training of the model with new data. Ensuring a secure and efficient process demands automated deployment and a dedicated CI/CD pipeline with DevOps integration, necessitating collaboration between technical experts and ML professionals (**CM21** [LR28]).

Deploying Deep Learning (DL) software on platforms like mobile devices or the cloud emphasizes the challenge of compatibility and reliability. Challenges include data extraction, inference speed on various platforms, and environment setup, all of which impact maintainability (**CM20** [LR29, 30]).

A fintech company's experience highlights the challenge of varying hardware and platform parameters that can affect the model behavior and result in errors. Moreover, the dynamic allocation of resources and the determination of when to scale down a system can be complex (**CM20** [LR27]).

Due to model complexity, large training datasets, and data sensitivity, the increasing difficulty of reproducibility in ML compounds maintainability issues. Though sharing source code and data has improved reproducibility, retraining models and data sharing remain challenging. Using large-scale models necessitates extensive resources and expert knowledge in cloud, security, networking, and distributed processing. Platforms like Kubernetes can enable scalable inference but aren't specifically designed to enhance reproducibility, ML workflow, and lifecycle processes. Further efforts are required to address these intertwined deployment and maintainability challenges in ML (**CM21** [LR72]).

5.2.12 Model Deployment Solution. Deploying deep learning models on mobile devices is an essential task for broader accessibility. Frameworks like TF Lite and Core ML are utilized to convert models into mobile-compatible formats. A common practice in this conversion is model quantization, which minimizes memory and computation needs (Chen et al. [LR29]). In addition to that, they found that the majority of deployment issues occur during the model conversion stage, with unsupported operations being the most common problem (**SM20** [LR29]). Chen et al. also provide strategies for addressing different deployment issues. These include targeted learning of required skills and improving the usability of documentation provided by framework vendors [LR30].

DLHub addresses scientific ML's unique challenges, such as model publication and compatibility across various computing resources. It offers model publication with specific metadata, persistent identifiers, and flexible control (**SM21** [LR33]).

Deployment integration involves infrastructure building, reusability, and collaborative work between researchers and engineers. This collaboration emphasizes version control, code reviews, and Implementing MLOps environments enables continuous integration and delivery through a CI/CD pipeline and continuous retraining through a CT pipeline, leading to a safe and repeatable path for ML model development, deployment, and updating (**SM19, SM21** [LR20, LR28]).

Orchestration platforms, including Kubernetes, facilitate scalable inference in cloud computing. Managing platforms like KFserver, Polyaxon, and Kubeflow, however, demands expertise in various areas such as cloud and security. MLC, a cloud-based platform, is designed to enhance the reproducibility and review processes in ML conferences. It's infrastructure agnostic and includes tools for easy transfer and publication of trained models (**SM19, SM21** [LR72]).

5.2.13 Model Governance Challenges. Governing ML models is a pronounced challenge that requires attention to robustness, generalisation, and compliance with specific functional requirements. The process of model verification, including requirement encoding, formal verification, and test-based verification, is complicated and necessitates precision to meet various demands (**CM22** [LR20]).

The development and management of ML models are intricate, facing issues such as maintainability challenges in reproducibility and large-scale execution (**CM23** [LR33]). The existing tools like Git are insufficient for effectively tracking changes in different elements of the models, leading to complexities in managing AI components (**CM23** [LR34]).

Operational challenges include data uncertainty, which poses risks to prediction reliability and demands well-defined methods for countering performance degradation (**CM23, CM24** [LR71]).

Table 10. Current Ecosystem Maintainability Challenges When Building ML Application

Challenge	Description of the challenges	ML stage	Paper
CM32	Proprietary AutoML libraries and systems.	AutoML	LR51, 60, 61, 62, 64
CM33	Gaps in AutoML services.	AutoML	LR63,80
CM34	Achieving AI software quality	ML System quality	LR10,57,56, 59, 73, 85,87
CM35	Human factors on ML model quality.	ML System quality	LR38,54, 75
CM36	Model deployment and integration	MLOPS	LR28, 52,53
CM37	Implementing MLOPS tools and workflows.	MLOPS	LR53, 74
CM38	Developing ML models.	Building ML system	LR10,11,49, 70
CM39	Deploying ML models in practice.	Building ML system	LR11,20, 27,47, 14,28
CM40	Maintaining deployed models.	Building ML system	LR27,51,50,31,48
CM41	Aligning ML models with long-term goals.	Building ML system	LR31,70

Current risk mitigation strategies are only partially effective, requiring revised risk management approaches for AI systems (**CM24** [LR73]).

The opacity of ML systems emphasizes the importance of data governance, and existing governance structures must be enhanced to manage societal issues related to AI. This includes addressing regulatory lag and understanding the technology, reflecting the need for further research into the roles and values in AI policies, and highlighting the continuous challenges of inclusivity and diversity in AI governance (**CM22, CM24** [LR88]).

5.2.14 Model Governance Solution. Machine learning's importance in research and industry necessitates standardized methods for various aspects of model development and deployment. DLHub addresses the challenges by facilitating model publication with precise metadata and adjustable access control (**SM23** [LR33]).

Risk assessment and compliance with regulations are vital, necessitating specialized teams. The process includes requirement encoding, formal verification, and test-based verification to satisfy business and regulatory needs (**SM24** [LR20]).

Mining Software Repository4ML (**MSR4ML**) automates the identification and tracing of artefacts in ML projects, enhancing productivity and traceability (**SM23** [LR34]).

Tools like MLFlow, NeptuneML, and WandB are being developed to manage ML-specific challenges, such as reproducibility and traceability. Refining engineering processes is ongoing (**SM23** [LR68]).

Steimers et al. suggest using three components of trustworthy AI to classify risk sources. These encompass lawful, ethical, and robust factors, addressing ethical aspects and reliability and robustness factors to enhance trustworthiness (**SM22, SM24** [LR73]).

Reproducibility in ML models is crucial but challenging due to the stochastic nature of ML training. Tracking and storing model information, using version control, and metadata handling can help ensure reproducibility (**SM23** [LR85]).

Lastly, governing AI through code-enforced rules is promising but challenging. Concrete specifications and identifying responsible government parties are required (**SM22** [LR88]).

5.3 RQ 3. Current Ecosystem Engineering Challenges and Solutions in Building an ML System or Applications

This section addresses RQ3, focusing on Current Ecosystem Engineering Challenges and Solutions in ML system development. Table 10 identifies primary challenges encountered in the current ecosystem, categorized according to different stages of ML development and referenced with relevant academic papers. Table 11 links each challenge to appropriate solutions, providing a range of

Table 11. Current Ecosystem Maintainability Solutions When Building ML Applications for the Identified Challenges in Table 10

Index	Solutions	Solution/Tools/frameworks/methods
CM32	SM32	AutoBazaar[LR51]. Meta-Learning, Auto-Weka , Auto-Sklearn and [LR60] , Auto-Keras [LR61],
CM33	SM33	AutoBazaar[LR51], Katib, Dataiku AutoML, H2O AutoML, Google AutoML, Ultron, Properteriay WALTS [LR80] , Auto-Keras [LR61]
CM34	SM34	MLCask [LR50] , CRISP-ML(Q)[LR85], QA4AI [LR85]
CM35	SM35	Recommendation from [LR75], CRISP-ML(Q) [LR85]
CM36	SM36	TensorFlow Extended (TFX), ModelOps, and Kubeflow [LR52]
CM37	SM37	CD4ML[LR52], MLFlow, Amazon SageMaker, Spack, and EasyBuild, [LR74]
CM38	SM38	DVC, MLFlow, Pachyderm, ModelDB, and Quilt Data [LR14], high-level ML programmings, such as WEKA, Azure Machine Learning Studio, KNIME, Orange, BigML, mljar, RapidMiner, Streamanalytix, Lemonade, and Streamsets [LR81], Technology Readiness Levels for ML [LR20], Weka, Orange, RapidMiner, and Knime, PowerBI, PyCaret, and Cloud AutoML [LR84]
CM39	SM39	Overton[LR25], Flame [LR31], ThunderML [LR49], MLCask[LR50],KOI [LR65], Looper [LR70], Cirrus[LR91]
CM40	SM40	TFX, KubeFlow, MLflow, H2O, Skymind Intelligence Layer, Uber's Michelangelo, Facebook's FBLearner Flow, Data Robot, Polyaxon, Comet, Atalaya, Amazon's SageMaker, and Microsoft Azure Machine Learning [LR48]
CM41	SM41	No good solution found in this SLR

tools, frameworks, and methods identified as effective in addressing the specific challenges, supported by references from the literature

5.3.1 AUTOML Challenges. **Automated Machine Learning** (**AutoML**) is an emerging field that searches for the best-performing model for a given task, dataset, and evaluation metric, reducing the human resources, and expertise required to develop machine learning models and decreasing the time-to-market for these models [LR80]. It uses heuristics and iteration to find the most accurate model for a task, and different types of AutoML software are available. However, it still requires understanding the underlying concepts [LR55]. In addition to that, AutoML also faces several challenges, including limitations of current AutoML libraries and systems, focused on specific use cases and being proprietary, time-consuming trial-and-error testing of ML algorithms and configurations, difficulties with network morphism and Bayesian optimisation in **Neural Architecture Search** (**NAS**) (**CM32** [LR51, 60, 61]). Additionally, the AutoML libraries and systems, such as ATM, Vizier, Rafiki, Google AutoML, DataRobot, and Azure Machine Learning Studio, have limitations, such as focusing on a specific subset of ML use cases and being designed as proprietary applications that do not support community-driven integration of innovations (**CM32** [LR51,62, 64]).

Bajaj et al. evaluate existing commercial AutoML services and found some gaps, such as the lack of customizability, transparency, and unique enhancements, as well as the use of only full-precision models (**CM33** [LR80]). Auto ML minimises expert intervention by autonomously selecting a model and algorithm, but data preprocessing automation is often overlooked (**CM33** [LR63]).

5.3.2 AutoML Solution. Auto-WEKA initiated the journey of AutoML solutions, focusing solely on classification, but it failed to handle missing data. Several other platforms followed, each with unique features and limitations. For example, Katib required user hints and may be complex for non-experts (**SM33** [LR80]). Machine Learning Bazaar streamlined the development of machine learning software by providing a unified framework, as demonstrated in real-world use cases and open-source variant AutoBazaar (**SM32, SM33** [LR51]).

Frameworks like Auto-WEKA, Auto-Sklearn, and SmartML have utilized meta-learning, with some proposed as microservice architectures for scalability (**SM32** [LR60]). Auto-Keras is a recent addition using network morphism and Bayesian optimization for neural architecture search (**SM32** [LR61]).

Specific platforms were developed for various applications, such as an autonomic platform for Smart City applications [LR62], and AutoTrain, focusing on small-scale image classification. The

AutoAugment and DeepAugment methods were proposed for data enhancement and augmentation respectively (**SM33** [LR63]).

ALOHA is a new tool to address the need for a more targeted approach to automating the deep learning pipeline [LR64]. Commercial applications include Walmart's WALTS, an enterprise-scale framework, and Ultron, limited to hyperparameter optimization, along with other commercial tools like Google Cloud AutoML, H2O AutoML, and Dataiku AutoML (**SM33** [LR80]).

5.3.3 Machine Learning System Quality Challenges. Maintaining and managing AI/ML systems present substantial challenges. These encompass software testing, model development, data management, project management, infrastructure, requirements engineering, and they further branch into subcategories like AI ethics, interpretability, and scalability. A notable concern is the absence of methods and tools to implement AI ethics, problems in consistently labeling large datasets, and the complexity in integrating ML models into scalable systems while ensuring data quality (**CM34** [LR10, 57]). Quality and reliability issues are also faced by DL software, particularly in feature engineering, model selection, and hyperparameter tuning (**CM35** [LR38, 54]). A knowledge gap persists in introducing fault tolerance and quality assurance in AI systems (**CM34** [LR56]), leading to hesitations in the business adoption of AI. Adapting current software architecture practices is necessary to address quality attributes such as explainability, verifiability, and privacy (**CM34** [LR59]).

The demand for human-centered approaches and compliance with safety principles is vital in AI's progress. This complexity can result in risks (**CM34** [LR73]), and therefore, building safe and trustworthy AI systems is essential. Some research suggests assurance cases for quality assurance and certification of AI applications, but concrete criteria are lacking. Risk management must adapt to the unique challenges posed by AI **CM34** [LR73]). Collaboration between software engineering and data science teams is critical, but additional challenges arise, such as the need for continuous monitoring and evolving quality requirements like explainability and fairness (**CM35** [LR75]).

Surveys reveal that practical ML projects must meet client expectations, with key challenges including data and software quality. A Japanese industry consortium, QA4AI, was formed to address these issues (**CM34** [LR85]). Understanding the impact of different platforms on deep learning software is essential for effectively developing and deploying high-quality systems. PyTorch, for instance, is noted for stable training and validation over other frameworks like TensorFlow, and there are discrepancies across frameworks leading to model misclassification. Care is required in model conversion between frameworks, and robustness against adversarial attacks varies between different deep learning models (**CM34** [LR87]).

5.3.4 ML System Quality Solution. Machine learning-based AI systems face the challenge of incomplete specification, a result of the complexity of the task and the operating environment. This complexity is seen as a source of risk, and a detailed taxonomy of risk sources has been developed to understand these risks and devise suitable mitigation measures (**SM34** [LR73]). Collaboration enhancement advises formalized practices, including the introduction of the CRISP-ML(Q) process model that aligns with risk-based thinking, and is designed to be universally applied (**SM35** [LR85]).

Quality aspects such as robustness, security, and data privacy should be considered in AI systems, along with tools like input/output checkers (**SM34** [LR56, 57]). Manual inspection and traditional testing, as well as seven leading software quality techniques for ML, including Deep Learning and Model Checking, are recommended (Santhanam et al. [LR58]; Masuda et al. [LR67]; [LR38]). Collaborative tools like MLCask have also been introduced to improve quality control and collaboration (**SM34** [LR50]).

The overarching goal is to foster collaboration among data scientists and engineers, requiring interdisciplinary teams, clear responsibilities, recognizing engineering as essential, and prioritizing process planning (**SM35** [LR75]). These comprehensive efforts aim at enhancing the quality control, collaboration, and standardization in ML development, providing a robust framework for future applications.

5.3.5 MLOPS Challenges. **Machine Learning Operations** (**MLOps**) applies DevOps principles to machine learning workflows, encountering challenges such as high costs, scarcity of data scientists, privacy concerns, and difficulties in model finalization and deployment (**CM36** [LR28]). Challenges also include experiment tracking, version control, and integration problems. **Continuous Delivery for Machine Learning** (**CD4ML**) offers an automation approach but faces obstacles in data operations and multi-organisational collaboration (**CM36** [LR52]). Research into ML platforms with DevOps capability is underway, with a focus on automation to alleviate bottlenecks such as GPU utilization (**CM36** [LR53]). Although tools like MLFlow, Amazon SageMaker, Spack, and EasyBuild are being developed to streamline ML workflows, they are still in their early stages and present practical challenges. Thus, further research is necessary to address the complexities of MLOps for more effective industry adoption (**CM37** [LR53,74]).

5.3.6 MLOPs Solution. John et al. present a systematic approach for the adoption of **Machine Learning Operations** (**MLOps**) within companies, featuring a maturity model that identifies four stages and has been validated through the examination of three case studies (**SM37** [LR28]). Zhou et al. [LR53] focus on machine learning platforms, offering solutions for orchestration, data management, and performance monitoring in hybrid environments. Granlund et al. [LR52] propose integration mechanisms to create multi-organizational AI/ML systems and present examples such as **Continuous Delivery for Machine Learning** (**CD4ML**) as a form of MLOps pipeline (**SM37** [LR52]). These integration mechanisms utilize platforms such as **TensorFlow Extended** (**TFX**), ModelOps, and Kubeflow, emphasizing orchestration, data storage, and management (**SM36** [LR52]). Rzig et al. [LR74] accentuate the merits of employing DevOps tools in ML projects, leading to improved code sharing, integration speed, and issue resolution, as well as the need for co-evolution analysis to facilitate early bug detection (**SM37** [LR74]). The benefits of adopting DevOps tools within ML projects are stressed, with a recommendation for data scientists and ML developers to utilize these tools in their work.

5.3.7 Building ML System Challenges. ML presents vast opportunities for real-world applications but also poses significant challenges in implementation and management. The success of ML models depends on ensuring reproducibility, which can be hindered by factors such as limited access to training data, lack of documentation, and inconsistencies in software and platforms (**CM38** [LR10, 11]).

Developing ML models entails addressing various issues like handling large inputs, ethical considerations, model complexity, accuracy, data limitations, uncertainty, and more. Acquiring enough labeled data for supervised learning remains a formidable task, demanding a profound understanding of interconnected steps like data preprocessing and model deployment (**CM39** [LR11, 20]). The challenges also encompass the entire lifecycle of an ML system, including data collection, monitoring, risk assessment, parallel experimentation, HPO, and collaboration among data scientists for version control and scalability (**CM39** [LR14, 27, 28]).

Deployment of ML models in real-world scenarios may be time-consuming and fraught with failure risks due to expertise shortage, data bias, and high costs. The growth of ML has triggered the emergence of versioning tools, but these must be in sync with other software artifacts [LR14].

A scarcity of tools exists for supporting the model lifecycle, highlighting the need for cost-effective maintenance and enhancement of deployed models (**CM40** [LR27]).

The complexity of building and deploying ML-based solutions requires a unified environment for prototyping, deployment, and maintenance at scale, involving advanced feature engineering and real-time data handling (**CM40** [LR48]). Collaboration issues in ML environments may arise, necessitating version control semantics and efficient storage (**CM40** [LR50]). The lack of complete systems for specific tasks can lead to intricate and fragile pipelines (**CM40** [LR51]).

Offering intelligent ML strategies to product engineers involves aligning ML's loss functions with the product's long-term goals, addressing deviations in data, understanding correlations versus causal links, A/B testing for performance assessment, and real-time feature extraction and inference. It also entails the need for simple, standardized, and future-proof ways to integrate innovative strategies into products (**CM38, CM41** [LR70]).

5.3.8 Building ML System Solution. The recent 25% YoY growth in machine learning has seen a shift from **Data-Oriented Architecture** (**DOA**) to streaming-based architecture, addressing micro-service data flow issues. Tools such as DVC, MLFlow, Pachyderm, ModelDB, and Quilt Data have emerged to assist data engineers and scientists in versioning data and models, with frameworks like TRL4ML supporting prioritization in ML projects (**SM38** [LR14]). Self-supervised, semi-supervised, and representation learning algorithms have been proposed for constructing data pipelines [LR24], and platforms like Overton facilitate deep learning application development (**SM39** [LR25]).

ML modeling in fintech applications requires consideration of governance, risk management, and compliance [27]. A framework has been proposed to support the ML model's entire lifecycle, addressing complexities through aspects such as Automated Data Quality Assurance, Domain Adaptation, and Hybrid Model Design [46, 47]. Tools like Flame offer advanced management features for model creation, along with platforms like TFX, KubeFlow, MLflow, and ThunderML for industrial AI model creation and deployment (**SM39** [LR31, LR48, LR49]). MLCask further provides versioning support for ML pipelines as an end-to-end analytics system (**SM39** [LR50]).

Algorithm selection for specific use cases can be time-consuming for non-experts [LR60], and architectures like KOI and Looper ML aim at simplifying this process (**SM39** [LR65]). Companies such as Google, Meta, LinkedIn, and Netflix are utilizing data-centric, vertical ML platforms that offer end-to-end solutions through high-level abstractions (**SM39** [LR70]). **Flow-Based Programming** (**FBP**) and **Model-Driven Software Development** (**MDSD**) offer further simplification, with products like WEKA, Azure Machine Learning Studio, and KNIME supporting high-level ML programming (**SM38** [LR81]).

Deploying ML projects on platforms like GitHub is challenging, but tools like Docker have been studied for their functionality in deploying open-source ML projects, encompassing aspects like data management, task scheduling, and model management (**SM38** [LR82]). CompareML provides a method to evaluate and compare ML libraries, enabling practitioners to make informed choices without extensive ML knowledge (**SM38** [LR84]).

Lastly, Cirrus is an innovative ML framework designed for serverless computation, utilizing features like ultra-lightweight worker runtime and stateless worker architecture to overcome serverless computing constraints. It aims to offer scalability for ML training and has demonstrated superior performance over both serverless and specialized ML frameworks (**SM39** [LR91]).

5.4 Scalability Challenges and Solutions

This section delves into scalability challenges and solutions in ML workflows, addressing Research Questions 4 through 6 (RQ4-RQ6). We identify specific scalability hurdles in Data Engineering

Table 12. Scalability Challenges in ML Workflows

Challenge	Description of the challenges	ML	Paper
CS01	Managing Data pipeline and its infrastructure for large-scale ML	DE / ME	LR92,95
CS02	Real-time analysis from large, diverse data requires new approaches.	DE	LR95,101
CS03	Sparse data and lack of metadata taxonomy create cold start challenges in large-scale online services.	DE / ME	LR101
CS04	Collecting and processing data on a large scale raises privacy and legal concerns.	DE	LR109,113
CS05	Configuring optimal hyperparameters is time-consuming.	ME/ CEC	LR99
CS06	Distributed model training and its drawbacks which diminishes parallel training advantages.	ME	LR116,123
CS07	Balancing and scheduling load across heterogeneous hardware can affect training speed and quality.	ME / CEC	LR95,96,101
CS08	Deploying and serving DL models requires modifying training pipelines and regular retraining.	ME	LR94,101
CS09	Training in constrained environments pose challenges, particularly with privacy-sensitive data.	ME / CEC	LR111,112.122
CS10	Understanding ML diagnosis, reproducibility, and diverse error sources are challenging.	ME / CEC	LR98.104
CS11	Managing infrastructure for large-scale ML is complex costly.	CEC / ME	LR106,107
CS12	Efficiently supporting ML workloads on clusters requires optimal hardware and software configurations.	CEC	LR94,97,107,120
CS13	Programmability and portability prioritisation in ML models and frameworks can lead to performance trade-offs.	CEC	LR110 , 92

(RQ4), Model Engineering (RQ5), and the Current Ecosystem (RQ6), and propose effective solutions.

Our findings are summarized in Tables 12 and 13. Table 12, "Scalability Challenges in ML Workflows," enumerates scalability issues in ML, each with a unique code (e.g., CS01). These are mapped to ML stages and supported by references.

Table 13, "Scalability Solutions for the Identified Challenges," links each challenge to a solution. Solutions are coded (e.g., SS01 for CS01) and comprise tools and methods addressing the challenges in Table 12, also supported by references.

5.5 RQ 4. Data Engineering Scalability Challenges and Solutions

5.5.1 Data Engineering Scalability Challenges. In the field of data engineering, scalability is paramount, facing distinct challenges that encompass a broad spectrum of considerations. The emerging need for exascale systems to handle large-scale data analysis, machine learning, and real-time applications represents a complex challenge, demanding new scalable data mining solutions, advanced analysis tools, and apps that operate on high-performance architectures (**CS01** [LR92]).

A significant concern arises with parallel data access, where increased data speeds come with the risk of fault resilience issues, an open and unresolved challenge (**CS01** [LR92]). The effectiveness of model training is contingent upon managing vast amounts of production data, scheduling, and monitoring, all of which present logistical difficulties (**CS07** [LR101]). Concurrent challenges include energy consumption, multitasking, scheduling, reproducibility, resilience, and the necessity for innovative programming constructs and runtime systems (**CS13** [LR92]).

Moreover, companies like Facebook are driven to create new parallel computing strategies to meet the intense computational demands posed by various machine learning models, a complex undertaking in itself (**CS01, CS02** [LR95]). The management of large-scale online services introduces further challenges such as cold start issues, sparse data, inadequate data structures, and the complex synthesis and aggregation of dispersed data (**CS03, CS02** [LR101]).

Table 13. Scalability Solution for the Identified Challenges in Table 12

Index	Solutions	Solution/Tools/frameworks/methods
CS01	SS01	DeLight [LR100], Data Mining Cloud Framework (DMCF), Data Analysis as a Service (DAaaS) [LR92]. Cylon [LR8]
CS02	SS02	Kafka [LR97] , Kafka-ML [LR121], Amazon Kinesis [LR121], Kafka Streams, Faust and Spark [LR121] Apache SAMOA, Apache Flink, Apache Spark streaming and TensorFlow Serving [LR121]
CS03	SS03	Warm starting initialises the new model with the neural network and weights from a previous model [LR101], Transfer Learning, and Self-supervised pre-training [LR118]
CS04	SS04	Kubric [LR109], CMA [LR102], Transfer Learning [LR118], Federated Learning (FL) and Split Learning (SL) [LR113]
CS05	SS05	Ultron-AutoML [LR110], Meta-learning to automate model selection and HPO [LR60]
CS06	SS06	Switch Transformer [LR115], ZeRO optimisation technique [LR116], DeepSpeed2 [LR116], and DeepSpeed-MoE [LR117]. Cloud providers, such as Google, Microsoft, Amazon, and IBM, are among the providers offering these services. [LR120], Kafka ML, Kubeflow, Sagemaker , Tensorflow Serving [LR121]
CS07	SS07	Elastic ML[LR119], Cloud providers offer ML as a service to abstract away the complexities and other synchronous and asynchronous parallelisation techniques [LR120]
CS08	SS08	Apache Mahout [LR94] and MLlib in HPC [LR112], Bighead [LR48], ASML [LR96], DLHub [LR119], TensorFlow Serving, SageMaker, Clipper [LR119], Kafka-ML [LR121],
CS09	SS09	Four layered edge-cloud architecture and recommendation from [LR112], Recommendation and Techniques from [LR114], Balancing workloads and reducing computation time using synchronous and asynchronous parallelisation techniques [LR120], Blind learning [LR113], FEDn [LR111], Transfer learning and self-supervised pre-training techniques [LR118].
CS10	SS10	Curious Container [LR98], ModelDB and ModelHub [LR119] , TensorFlow debugger [LR101]
CS11	SS11	Curious Container [LR98], A framework for microservice architecture for training and managing model [LR99], DistRDF2ML [LR104], ModelDB and ModelHub [LR119], Amazon SageMaker [LR124], TensorFlow Serving, SageMaker, Clipper, and DLHub [LR119]
CS12	SS12	ADCME[LR93], IBM Fabric for Deep Learning (FfDL), Djinn, Tonic, Ray, and Ease.ml [LR119], Amazon SageMaker [LR124], Hybrid distributed architecture showcasing compatibility with multiple systems without code changes [LR105]
CS13	SS13	OpenSHMEM-ML [LR110]

Lastly, the challenges related to the quality and quantity of training data are profound. Collecting large-scale real data is fraught with obstacles such as cost, privacy, and legal concerns (**CS04** [LR113]). Synthetic data, although promising as a cost-effective solution, is hindered by immature generation tools, contributing to fragmented efforts (**CS04** [LR109]).

5.5.2 Data Engineering Scalability Solutions. The proliferation of large-scale data analysis has encouraged the use of cloud systems, leading to the inception of the **Data Analysis as a Service** (**DAaaS**) model. This model includes three sub-models: DAIaaS, DAPaaS, and DASaaS, which foster the structured creation of extensive data analysis systems like the **Data Mining Cloud Framework** (**DMCF**) (**SS01** [LR92]).

Frameworks like Delight are instrumental for deep learning in constrained environments, offering adaptive learning and customization to enhance efficiency (**SS01** [LR100]). Additionally, Cylon, a distributed memory data parallel library, demonstrates high performance for structured data processing (**SS01** [LR8]). Technologies like Kafka, Kafka-ML, and Amazon Kinesis are vital for real-time data analysis and integration with ML/AI (**SS02** [LR97, LR121]).

Challenges in large-scale online services include cold start and sparse data issues. Solutions such as incremental training, warm starting, and surrogate tasks help to address these challenges (**SS03** [LR101]). Transfer learning and self-supervised pre-training have emerged as solutions for data-hungry DNNs, including for tasks like NLP. Efficiency in large-scale models can be improved through optimization, efficient algorithms, and model compression (**SS03** [LR118]).

Collecting and processing large-scale data presents issues like cost, complexity, and privacy concerns [LR113]. Techniques such as **Federated Learning** (**FL**) and **Split Learning** (**SL**) allow shared model training without data exchange (**SS04** [LR113]). Additional solutions include synthetic data, transfer learning, and stringent validation and verification processes. An example is

Kubric, a Python framework that generates realistic scenes, and the CMA system, which validates image data and verifies tools and algorithms. Implementing these strategies ensures accurate results while minimizing risks in sensitive fields like healthcare and biology (**SS04** [109, 102, 118]).

5.6 RQ 5. Model Engineering Scalability Challenges and Solutions

5.6.1 Model Engineering Scalability Challenges. Scalability in Model Engineering is a complex issue that includes various challenges, such as data scaling, model size, and count. These complications result in problems like cold start, sparse data, and balancing efficiency and effectiveness, among others. They further highlight the need for continuous retraining and improved infrastructure design [LR101]. There is no standardized deployment method due to the case-specific nature of scalability in architectures and application frameworks [LR95,96]. Understanding vast datasets and discerning patterns for non-technical users are increasingly challenging, along with the intricacies of extensive data infrastructure (**CS05** [LR99]).

Distributed training involves dividing the neural network graph or dataset into multiple parts to be trained on multiple machines. This requires high-performance resources to reduce communication and memory costs (**CS09** [LR101]). Training DNNs is a time-consuming process, and deploying tasks on multiple nodes can cause communication overhead, becoming a bottleneck. Communication overhead remains an issue despite high-performance hardware accelerators (**CS06** [LR116, LR123]).

Optimizing hardware and software configurations for ML workloads on HPC clusters is challenging. A study on TensorFlow workflow emphasized the need for optimized libraries tailored for specific architectures and performance dependencies (**CS11** [LR107]). On-device learning presents its difficulties, such as insufficient user data and limitations in processing speed, requiring further research (**CS09** [LR106]).

Managing data traffic in distributed ML training is vital to prevent network saturation. Techniques like compression and scheduling algorithms are essential but bring challenges in load balancing and synchronization (**CS07** [LR95,96,101]). There is a shift toward executing ML workloads on modern hardware like GPUs [LR94], yet deploying research-generated ML models is challenging due to differences between frameworks (**CS08** [94, 101]).

Long-term viability in distributed ML depends on addressing performance, fault tolerance, privacy, and portability. While federated ML is progressing, open frameworks that effectively tackle distributed computing challenges are scarce (**CS09** [LR111]). Edge computing in distributed ML adds constraints like limited capacity, energy conservation, and data privacy, with communication overhead being a significant obstacle (**CS09** [LR112]).

The modern statistical ML field experiences a constant struggle between accuracy and efficiency. This balance is particularly delicate in real-time distributed ML systems in autonomous vehicles [LR122]. Challenges are even more pronounced in ML systems for innovative services due to IoT devices' limitations. Achieving scale in distributed ML operations for edge intelligence is essential but intricate due to the IoT's heterogeneous and dispersed nature, necessitating careful consideration of energy efficiency, privacy, and real-time inference requirements (**CS09** [LR122,123]).

5.6.2 Model Engineering Scalability Solution. Shahoud et al. developed a scalable framework that automates model selection through meta-learning in Big Data, emphasizing time series models [LR60]. Ultron-AutoML, an open-source HPO framework, focuses on user-friendliness and efficiency (**SS05** [LR60, LR110]).

Several parallel frameworks like Apache Mahout [LR94] and MLlib [LR112] are available in the HPC ecosystem, providing scalable machine learning algorithms. Bighead is a cost-effective ML

platform offering end-to-end solutions [LR48], and the ASML framework targets standardized ML pipeline deployment (**SS08** [LR96, LR119]).

Transfer learning and self-supervised pre-training can improve efficiency in DNNs (**SS09** [LR118]). The complexity of deep learning models requires hardware and framework support for training and inference acceleration, such as GPUs, FPGAs, TPUs, and parallelization strategies (**SS06** [LR103]). Techniques like the Switch Transformer model and ZeRO optimization are employed to reduce computational costs and improve performance [LR115, LR116]. DeepSpeedMoE offers an end-to-end training and inference solution for massive MoE models, reducing inference latency and cost (**SS06** [LR117]).

Cloud providers like Google, Microsoft, Amazon, and IBM facilitate ML adoption by offering machine learning as a service, making it accessible to a broader audience (**SS06** [LR120]). Challenges in deep learning scheduling are addressed through various algorithms and elastic ML frameworks, with cloud providers offering pre-packaged ML software to simplify development (**SS07** [LR119, LR120]).

Kafka-ML is a framework supporting data streams integration with ML/AI pipelines, and it offers a UI for training and inference, compatible with frameworks like TensorFlow (**SS08** [LR121]). Enhancements in distributed deep learning come through architectural approaches, optimizations, gradient compression, and infrastructure strategies (**SS06, SS09** [LR114]). Balancing workloads for efficient distributed machine learning requires various using synchronous and asynchronous parallelisation techniques (**SS07, SS09** [LR120]).

Blind Learning (**BL**) provides an automated software framework for distributed deep learning, focusing on the challenges of centralized data sharing in healthcare (**SS09** [LR113]). FEDn is a scalable federated learning framework designed for real-world and industrial applications [LR111]. A four-layer edge-cloud continuum architecture proposes solutions for executing ML algorithms in distributed IoT environments, emphasizing load balance, fault tolerance, and a bagging-based ensemble learning algorithm for image classification [LR112]. Considerations in decentralized algorithms on IoT include data location, distribution, task scheduling, and coding and testing.

5.7 RQ 6. Current Ecosystem Scalability Challenges and Solutions

5.7.1 Current Ecosystem Engineering Challenges. Amidst rapidly evolving technology, the challenges of scalability in the current ecosystem are multifaceted. One of the main concerns is integrating support for ML workloads on **High-Performance Computing** (**HPC**) clusters, which requires optimal hardware and software configurations (**CS12** [LR107]). Distributed ML is gaining traction but faces unresolved challenges such as performance, fault tolerance, and privacy (**CS12** [LR120]). Traditional systems like Hadoop have limitations, thus shifting focus towards platforms like TensorFlow and PyTorch, designed for ML and compatible with modern hardware like GPUs (**CS12** [LR94]). Real-time analysis and prediction require novel approaches and options like Apache Kafka and Spark Streaming are popular, although limited in ML usage (**CS12** [LR97]). Python's interpreted nature poses performance issues, resolved by adding highly performant code in C or C++ [LR110]. Large-scale ML/DL (Deep Learning) implementation faces complexities in cost efficiency, algorithm and data costs, tensor-based mismatches, and transfer to production and mobile deployment [LR95, **CS11** [LR106]]. Cloud computing presents privacy and resource cost challenges, and on-device learning, although a solution, still needs advancements (**CS12** [LR106]). Managing large datasets in ML requires substantial infrastructure, often leading to costly cloud resources (**CS11** [LR108]). Finally, ensuring reproducibility in large-scale ML systems is problematic due to data complexity, platform dependencies, and the limited understanding of ML diagnosis (**CS10** [LR98, LR104]).

5.7.2 Current Ecosystem Engineering Solutions. The current ecosystem of engineering solutions in machine learning consists of various technologies and frameworks. The ONNX (Open Neural Network Exchange) offers interoperability between machine learning frameworks and libraries (**SS13** [LR95]). Container technologies like Docker and the Curious Containers project enhance reproducibility [LR95, LR98]. Shahoud et al. have explored frameworks for training and managing ML models (**SS11** [LR99]). Model management is addressed by ONNX, ModelDB, and ModelHub, with DistRDF2ML providing explainable ML pipelines (**SS10, SS11** [LR119, LR104]). TensorFlow has been highlighted for its ease of use and debugging capabilities, while ADCME and Apache Spark are noted for distributed computing (**SS10, SS12** [LR93, LR101]). Mayer et al. cover deep learning infrastructure, including hardware components and tools like IBM Fabric for Deep Learning (**SS12** [LR119]). Amazon SageMaker's streaming model is praised for its scalability and efficiency (**SS11, SS12** [LR124]). Tasgetiren et al. have proposed a hybrid architecture for the banking industry (**SS12** [LR105]). Various approaches for high-performance data science and machine learning frameworks exist, including Legate Numpy and Arkouda, which consider productivity and performance [LR110]. SHMEM-ML provides efficient distributed training performance, leveraging HPC software stack and interoperating with the Python machine learning ecosystem (**SS13** [LR110]).

6 Discussion

As ML systems grow in complexity and become more pervasive, striking a balance between scalability and maintainability has emerged as a critical challenge in ML development. The relationship between scalability and maintainability in ML systems is multifaceted and often involves complex tradeoffs. These tradeoffs are manifested in several interconnected ways, influenced by few overarching factors as follows:

Resource Constraints: Both scalability and maintainability are fundamentally limited by computational, human, and financial resources. Scaling up an ML model often demands significant computational power, which can complicate maintenance tasks like debugging or retraining. Bharti (2025) illustrates how optimizing real-time recommendation systems for scalability requires balancing resource allocation with maintainability considerations [3].

Temporal Dependencies: Scalability often focuses on immediate performance under high loads, while maintainability emphasizes long-term adaptability and ease of updates. These differing temporal priorities can conflict during system design. Microsoft's engineering playbook highlights the need to balance short-term scalability goals with long-term maintainability [27].

Feedback Loops: Changes made to improve one dimension (scalability or maintainability) often create feedback loops affecting the other. For instance, optimizing for scalability through distributed computing may necessitate more sophisticated maintenance protocols, increasing complexity [16, 29]. Poorly managed feedback loops can exacerbate issues in ML system design [42].

Based on our SLR and research questions, we identified and synthesized specific key interplaying tradeoffs between scalability and maintainability.

6.1 Specific Tradeoffs in Achieving Scalable and Maintainable ML System

Complexity vs Simplicity: Complex ML systems offer high accuracy but are harder to maintain and scale [LR33][LR34][LR35], aligning with the Resource Constraints and Feedback Loops factors. The demand for handling large datasets and real-time processing adds to this complexity [LR33]. Simpler models, like shallow networks or decision trees, are easier to manage and deploy, though they may sacrifice some performance [LR20][LR35]. Deep learning, while powerful, is not always optimal due to computational cost (highlighting Resource Constraints) [LR35]. The right balance

depends on application needs, resources, team expertise, and the environmental impact of energy consumption [LR18][LR20][LR35], reflecting the broader Temporal Dependencies.

Speed vs Accuracy: ML development faces a fundamental tradeoff between speed (training/inference) and accuracy [LR112], influenced by Resource Constraints. Faster training often reduces accuracy; higher accuracy requires more time. Solutions include hyperparameter optimization (e.g., Google Vizier, Amazon SageMaker, SigOpt) [LR21], distributed learning (with communication overhead challenges, impacting Feedback Loops) [LR120], and transfer learning (leveraging pre-trained models, addressing Resource Constraints) [LR118]. Using simpler models initially can accelerate deployment [LR20]. The choice between retraining and incremental modeling also affects this trade-off, with implications for Temporal Dependencies [LR18].

Scalability vs Performance: Increased scalability (e.g., distributed workloads, larger batches) can decrease individual computation speed [LR60][LR110], a direct consequence of Resource Constraints. This is evident in large-scale online services with issues like cold starts and sparse data [LR101]. Frameworks like Apache Mahout, MLlib [LR112], Bighead [LR48], and ASML [LR96] offer scalable solutions. Transfer learning and self-supervised pre-training address challenges with large annotated datasets (relating to Resource Constraints) [LR118]. Research focuses on system-level optimization, efficient algorithms, and model compression for large pre-trained models, acknowledging Resource Constraints and aiming to mitigate negative Feedback Loops [LR118].

Performance vs Resource Consumption: Higher ML performance often demands more computational resources and higher costs [LR112], a core aspect of Resource Constraints. This is particularly true in distributed systems, where resource management must balance performance and reliability. Solutions include subsampling/minibatches [LR122], low-precision representations [LR122], hardware accelerators (GPUs, FPGAs, TPUs) [LR120], and cloud-based delivery [LR120]–all attempting to manage Resource Constraints. For resource-constrained devices (e.g., IoT), computation offloading trades speed and privacy for resource savings [LR122], a clear example of conflicting Temporal Dependencies.

Flexibility vs Stability: Flexible ML systems adapt to changing needs, improving maintainability, but potentially reducing stability. Stable systems offer consistent performance but may be less adaptable. This tension reflects Temporal Dependencies. Balancing these requires automated deployment [LR95], dedicated CI/CD pipelines [LR28], standardized model packages [LR95], and reproducibility practices (tracking model information, code, data, and environments) [LR85][LR88]. Tools like ONNX (interoperability) [LR95], Docker (containerization) [LR95], and DistRDF2ML (pipeline orchestration) [LR104] aid in this balance, mitigating negative Feedback Loops.

Scalability vs Cost: Increased scalability, while enhancing performance and reducing downtime, often raises costs, especially with cloud resources [LR101], directly tied to Resource Constraints. Distributed learning offers benefits (time/energy savings, reliability) but introduces challenges (communication overhead, portability, privacy) [LR112][LR116], creating Feedback Loops. Edge device implementation faces further limitations (computing/storage, privacy, bandwidth) [LR112], highlighting Resource Constraints. Large cluster scales can lead to exponential communication overhead [LR116], a negative Feedback Loop. Solutions include synthetic data and transfer learning (reducing data needs and thus Resource Constraints) [LR124], cloud-based ML services [LR120], and tools like Amazon SageMaker (scalable training) [LR124].

The journey towards truly scalable and maintainable ML systems is ongoing, requiring continuous innovation and collaboration across the ML and software engineering communities. By

tackling these challenges head-on, we can unlock the potential of ML technologies and ensure their responsible and effective deployment in solving complex real-world problems.

7 Conclusion

In this comprehensive study, we have meticulously identified and catalogued 41 maintainability challenges and 13 scalability challenges within ML systems. This work serves as a valuable resource, offering a detailed examination of these challenges and their potential solutions across the domains of Data and Model Engineering and the Current ML ecosystem.The synthesis of these challenges is of paramount importance. It provides a nuanced understanding of the intricate relationship between maintainability and scalability, a relationship that becomes increasingly critical as ML systems and datasets continue to grow in complexity.

This study is not merely an academic exercise; it is a fundamental guide for practitioners and researchers alike who are navigating the complex landscape of ML system development and management.

References

[1] Saleema Amershi, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. 2019. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 291–300.

[2] Anders Arpteg, Björn Brinne, Luka Crnkovic-Friis, and Jan Bosch. 2018. Software engineering challenges of deep learning. In *Euromicro Conference on Software Engineering and Advanced Applications (SEAA'18)*. IEEE, 50–59.

[3] M. Bharti. 2025. Architectures and optimization strategies for real-time machine learning recommendation systems: A systematic review of scalability challenges. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* (2025). Forthcoming article. Volume, number, and page information not yet available.

[4] Megha Bhushan, Akkshat Pandit, and Ayush Garg. 2023. Machine learning and deep learning techniques for the analysis of heart disease: A systematic literature review, open challenges and future directions. *Artificial Intelligence Review* 56, 12 (2023), 14035–14086.

[5] Censius AI. 2025. ML Scalability. Censius AI Wiki, Retrieved from https://censius.ai/wiki/ml-scalability. Accessed: April 28, 2025.

[6] Conceptatech. 2023. The Importance of Scalability In Software Design. Retrieved from https://www.conceptatech.com/blog/importance-of-scalability-in-software-design. Accessed: March 17, 2023. Original publication date not specified.

[7] Ania Cravero et al. 2022. Challenges to use machine learning in agricultural big data: A systematic literature review. *Agronomy* 12, 3, Article 748 (2022).

[8] Malinda Dilhara, Ameya Ketkar, and Danny Dig. 2021. Understanding software-2.0: A study of machine learning library usage and evolution. *ACM Transactions on Software Engineering and Methodology* 30, 4, Article 55 (Oct. 2021), 42 pages.

[9] T. Dyba and T. Dingsøyr. 2008. Empirical studies of agile software development: A systematic review. *Information and Software Technology* 50, 9-10 (2008), 833–859.

[10] H. Edison, X. Wang, and K. Conboy. 2022. Comparing methods for large-scale agile software development: A systematic literature review. *IEEE Transactions on Software Engineering* 48, 8 (1 Aug. 2022), 2709–2731.

[11] G. Giray. 2021. A software engineering perspective on engineering machine learning systems: State-of-the art and challenges. *Journal of Systems and Software* 180, Article 111031 (2021).

[12] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning*. MIT Press. http://www.deeplearningbook.org

[13] K. A. Hallgren. 2012. Computing inter-rater reliability for observational data: An overview and tutorial. *Tutorials in Quantitative Methods for Psychology* 8, 1 (2012), 23–34.

[14] Chip Huyen. 2022. *Designing Machine Learning Systems*. O'Reilly Media, Inc.

[15] Institute of Electrical and Electronics Engineers. 1998. *IEEE Standard for Software Maintenance*. Standard IEEE Std 1219-1998. Institute of Electrical and Electronics Engineers. 1–56 pages. DOI : https://doi.org/10.1109/IEEESTD.1998.88278

[16] InterviewNode. 2024. Architecting Scalable ML Systems: A Guide for Senior Engineers by InterviewNode. Retrieved from https://www.interviewnode.com/. Accessed: December 17, 2024. Original publication date not specified.

[17] Christian Kastner. 2025. Machine learning in production: From models to products. MIT Press.
[18] B. Kitchenham. 2004. *Procedures for Performing Systematic Reviews*. Technical Report TR/SE-0401. Keele University, Keele, UK.
[19] Ask Berstad Kolltveit and Jingyue Li. 2023. Operationalising machine learning models: A systematic literature review. In *Proceedings of the 1st Workshop on Software Engineering for Responsible AI (SE4RAI'22)*. Page numbers not provided in the source. The workshop was in 2022, proceedings published in 2023.
[20] Klaus Krippendorff. 2018. *Content Analysis: An Introduction to its Methodology*. Sage Publications.
[21] Marco Kuhrmann, Daniel Méndez Fernández, and Maya Daneva. 2017. On the pragmatic design of literature studies in software engineering: an experience-based guideline. *Empirical Software Engineering* 22 (2017), 2852–2891.
[22] J. R. Landis and G. G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics* 33, 1 (1977), 159–174.
[23] Chao Liu, Cuiyun Gao, Xin Xia, David Lo, John Grundy, and Xiaohu Yang. 2022. On the reproducibility and replicability of deep learning in software engineering. *ACM Transactions on Software Engineering and Methodology* 31, 1, Article 15 (Jan. 2022), 46 pages.
[24] L. E. Lwakatare, A. Raj, J. Bosch, H. H. Olsson, and I. Crnkovic. 2019. A taxonomy of software engineering challenges for machine learning systems: An empirical investigation. In *Agile Processes in Software Engineering and Extreme Programming. XP 2019*, P. Kruchten, S. Fraser, and F. Coallier (Eds.). Lecture Notes in Business Information Processing, Vol. 355. Springer.
[25] Ruchika Malhotra and Anuradha Chug. 2012. Software maintainability prediction using machine learning algorithms. *Software Engineering: An International Journal (SeiJ)* 2, 2 (2012).
[26] R. Malhotra and A. Chug. 2016. Software maintainability: Systematic literature review and current trends. *International Journal of Software Engineering and Knowledge Engineering* 26, 8 (2016), 1221–1253.
[27] Microsoft. 2024. Scalability - Engineering Fundamentals Playbook. https://microsoft.github.io/code-with-engineeringplaybook/non-functional-requirements/scalability/
[28] T. Mikkonen, J. K. Nurminen, M. Raatikainen, I. Fronza, N. Mäkitalo, and T. Männistö. 2021. Is machine learning software just software: A maintainability view. In *Software Quality: Future Perspectives on Software Engineering Quality*. Software Quality, Vol. 404. Springer International Publishing, 94–105.
[29] ML-Architects. 2024. System Design and Continuous Delivery. Retrieved from https://ml-architects.ch/. Accessed: December 27, 2024. Original publication date not specified.
[30] ml-ops.org. 2023. ml-ops.org. Retrieved from https://ml-ops.org/. Accessed: March 17, 2023. Original publication date not specified.
[31] A. Munappy, J. Bosch, H. H. Olsson, A. Arpteg, and B. Brinne. 2019. Data management challenges for deep learning. In *2019 45th Euromicro Conference on Software Engineering and Advanced Applications (SEAA'19)*. IEEE, 140–147.
[32] Nadia Nahar, Shurui Zhou, Grace Lewis, and Christian Kästner. 2022. Collaboration challenges in building ML-enabled systems: Communication, documentation, engineering, and process. In *Proceedings of the 44th International Conference on Software Engineering (ICSE'22)*. Page numbers not provided in the source.
[33] E. d. S. Nascimento, I. Ahmed, E. Oliveira, M. P. Palheta, I. Steinmacher, and T. Conte. 2019. Understanding development process of machine learning systems: Challenges and solutions. In *2019 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM'19)*. Porto de Galinhas, Brazil, 1–6.
[34] David O'Brien et al. 2022. 23 shades of self-admitted technical debt: An empirical study on machine learning software. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'22)*. Page numbers not provided in the source.
[35] R. F. Paige, J. Cabot, and N. A. Ernst. 2017. Foreword to the special section on negative results in software engineering. *Empirical Software Engineering* 22, 5 (2017), 2453–2456.
[36] Paraskumar Patel. 2023. Challenges and solutions in cross-platform ML systems integration. *Journal of Artificial Intelligence and Cloud Computing* 2, 4 (2023), 1–5.
[37] T. Saracevic. 1995. Evaluation of evaluation in information retrieval. In *Proceedings of 18th Annual International Conference on RDIR*. 138–146. The conference acronym "RDIR" might be a typo, possibly for SIGIR.
[38] D. Sculley, Gary Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, and Dan Dennison. 2015. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett (Eds.). Curran Associates, Inc., 2503–2511.
[39] Alex Serban, Koen van der Blom, Holger Hoos, and Joost Visser. 2020. Adoption and effects of software engineering best practices in machine learning. In *Proceedings of the 14th ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM'20)*. Page numbers not provided in the source.
[40] K. Shivashankar and A. Martini. 2022. Maintainability challenges in ML: A systematic literature review. In *2022 48th Euromicro Conference on Software Engineering and Advanced Applications (SEAA'22)*. IEEE, Gran Canaria, Spain, 60–67.

[41] Christina Silver and Ann Lewins. 2014. *Using Software in Qualitative Research: A Step-by-Step Guide*. Sage Publications. DOI: https://doi.org/10.4135/9781473906907
[42] Trinetix. 2025. Overcoming Pitfalls in ML System Design. Trinetix Insights, Retrieved from https://www.trinetix.com/. Accessed: January 21, 2025. Original publication date not specified.
[43] Rajesh Vasa. 2025. Microservices vs. Monolithic Architectures: Strategic Trade-Offs in the AI Era. *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* 11, 1 (February 2025), 2428–2436. https://doi.org/10.32628/CSEIT251112269
[44] S. Wang et al. 2023. Machine/deep learning for software engineering: A systematic literature review. *IEEE Transactions on Software Engineering* 49, 3 (1 March 2023), 1188–1231.
[45] Yasuhiro Watanabe et al. 2021. Preliminary literature review of machine learning system development practices. In *2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC'21)*. IEEE. Page numbers not provided in the source.
[46] J. Webster and R. T. Watson. 2002. Analysing the past to prepare for the future: Writing a literature review. *MIS Quarterly* 26, 2 (2002), xiii–xxiii.
[47] C. Wohlin. 2014. Guidelines for snowballing in systematic literature studies and a replication in software engineering. In *Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering (EASE'14)*. ACM, 1–10.
[48] C. Wohlin, P. Runeson, P. A. da Mota Silveira Neto, E. Engström, I. do Carmo Machado, and E.S. de Almeida. 2013. On the reliability of mapping studies in software engineering. *Journal of Systems and Software* 86, 10 (2013), 2594–2610.
[49] Jeff Jun Zhang, Kang Liu, Faiq Khalid, Muhammad Abdullah Hanif, Semeen Rehman, Theocharis Theocharides, Alessandro Artussi, Muhammad Shafique, and Siddharth Garg. 2019. Building robust machine learning systems: Current progress, research challenges, and opportunities. In *Proceedings of the 56th Annual Design Automation Conference 2019*. 1–4.

Received 17 January 2024; revised 1 May 2025; accepted 3 May 2025